

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	I want to do a GSoC project with Oppia because I genuinely enjoy working on Oppia's codebase and mission. Over the last year, I have been contributing regularly to Oppia as part of the LEAP team, and during this time I've worked on many learner-facing issues, frontend fixes, backend changes, tests, and migration-related tasks. Through these contributions, I've developed a strong understanding of how Oppia's platform works and how even small improvements can have a meaningful impact on learners. Oppia serves millions of learners globally, and improving the Topic Page directly affects how confidently learners begin their journey, especially first-time users. The Topic Page is one of the first places learners interact with when starting a topic, and improving how lessons, practices, and revision content are presented can directly improve the learning experience. GSoC gives me the opportunity to work full-time on such an impactful feature, take ownership of it, and ensure that it is well-designed, tested, and maintainable.
What do you hope to learn/achieve during GSoC?	During GSoC, I want to improve my ability to design and implement larger features end-to-end, rather than working only on isolated issues. While contributing so far, I have mostly focused on fixing bugs, improving UI behavior, adding tests, and making responsiveness changes. Through this project, I want to learn how to plan, build, and ship a complete feature in a structured and maintainable way. Specifically, I hope to: • Gain deeper experience working on complex frontend features involving state management, responsiveness, and accessibility • Improve my ability to write clear and reliable acceptance tests for complex user flows • Develop stronger communication and collaboration skills by regularly discussing design decisions, progress, and trade-offs with mentors and other contributors • Learn how to incorporate feedback effectively and iterate on implementation based on reviews Overall, I want to come out of GSoC as a more confident and independent engineer who is comfortable taking ownership of larger responsibilities while collaborating effectively within an open-source team.

Which Oppia teams have you collaborated with, and what have you done on those teams?	LEAP	I have been contributing to the LEAP team for over one year. My work includes fixing learner-facing UI and responsiveness issues, implementing and updating Angular components, writing TypeScript and Python unit tests, adding and maintaining acceptance and e2e tests, debugging CI failures, working on datastore-affecting changes, and implementing Beam jobs. I have also contributed to larger refactors such as object factory migrations and worked on topic, dashboard, and learner-related features.
	Non-coding Contributions	- Assisted with issue triage and identifying root causes for frontend and full-stack bugs - Helped new contributors set up the local development environment, including running the development server and acceptance tests - Guided new contributors in debugging issues by sharing context about Oppia's codebase, common pitfalls, and testing workflows
Contact information	Email: Kartik Suryavanshi GitHub: https://github.com/KartikSuryavanshi	
Preferred method of communication	Email and GitHub discussions. I am also available on Google Chat .	
Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>	GitHub: https://github.com/KartikSuryavanshi LinkedIn: https://www.linkedin.com/in/kartiksuryavanshi	
Which timezone will you primarily be in during the summer?	Indian Standard Time (IST) - UTC+5:30	
(If you are a student) When are your school holidays?	I am currently in my final year of BTech. My college allows internships during this period, and there are no regular classes or assignments during the final year.	
What other obligations might you need to work around during the summer?	I do not have any other jobs, internships, exams, or major commitments planned between May 1 and September 30.	
Planned time commitment	I plan to dedicate around 15–20 hours per week to the GSoC project during the program period. Since I will not have parallel academic coursework, I will be	

	able to maintain a consistent schedule on weekdays, with flexibility for mentor meetings, reviews, and iteration based on feedback. I will make sure to plan my work in a sustainable way and communicate proactively if any issues or delays come up.
--	--

Project Timeframe

Note: Oppia will only be offering a single GSoC coding period timeframe this year, starting on **May 25**. All work for Milestone 1 must be completed and submitted by **Jun 26** for internal feedback (with any revisions due by **Jul 10**), and all work for Milestone 2 must be completed and submitted by **Aug 19** for internal feedback (with any revisions due by **Sep 2**). We will not be able to extend these deadlines.

Coding period	I will adhere to the above deadlines.
----------------------	---

Communication Channels

Note: The Oppia team places a high emphasis on communication, and we have found that daily contact between contributors and mentors is important for helping keep projects on track. This is why we ask that contributors send short daily updates to the team group chat explaining what they have done, where they are stuck, and what they plan to do next.

I can commit to sending daily updates to the team group chat, each day I work during the GSoC period.	Yes
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	In addition to daily email updates, I plan to communicate with my mentor through GitHub discussions or Google Chat as needed. I will also be available for weekly sync meetings via Google Meet to discuss progress, blockers, and next steps.

Contributor Briefing

Note: A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 1.	Yes
---	---

Section 2: About Your Project

Project Details

Project title <i>(should match the one on the Project Ideas list)</i>	Re-design the topic page
Project size <i>(should match the options on the Project Ideas list)</i>	Medium (~175 hours)
Why did you choose this project?	I chose this project because it closely aligns with the type of work I have been doing in Oppia over the past year and the areas where I am most comfortable contributing. Through my previous contributions, I have worked extensively on learner-facing pages, UI and responsiveness issues, scroll behavior bugs, and acceptance tests, many of which relate directly to the Topic and learner dashboard pages. The Topic Page plays an important role in guiding learners through lessons, practice sessions, and revision content. From my experience working on related issues, I have seen how small UI and state-related problems can make it harder for learners to understand what to do next. This project addresses those exact problems by integrating lessons, practices, and revision content in a clearer and more intuitive way. I was also drawn to this project because it feels like a natural next step from my existing contributions and gives me the opportunity to take ownership of a well-scoped, user-facing feature while continuing to work within parts of the codebase I already understand well.

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc . ¹	 Debugging Doc for #24268 Progress not updating issue
I can debug and fix CI failures	#25737

¹ To develop this skill: Tackle an issue where the root cause is unclear, identify the root cause in a comment on the issue thread (perhaps with a debugging doc explaining how you figured it out) and submit a PR that fixes it. Fixing CI failures also counts. You might find [this tutorial](#) helpful.

/ flakes. ²	#25600 #23618 #25178
WEB	
I can write Python code with unit tests. ³	#24724
I can write TS + Angular code with unit tests. ⁴	TypeScript + Angular with unit tests #24724 #21862 #25177 UI-specific Contributions These PRs demonstrate attention to UI behavior, responsiveness, and edge cases: #21495 #21442 #23328 #21330 #21334
I can write or modify acceptance tests. ⁵	#24083 #22137
I can write or modify Beam	#24275

² To develop this skill: See issues labelled “[CI breakage](#)” on GitHub, or look at [CI failures in develop](#) (or  signs [here](#)) and figure out how to fix them. [This tutorial](#) (for Web) might also be useful.

³ To develop this skill: Some [LaCE](#) and [Contributor Dashboard](#) issues have a backend component, and many [Dev Workflow](#) issues do too. Focus on issues that aren’t marked as “backlog”. This is also a good issue to gain experience with backend tests: [#21308](#).

⁴ To develop this skill: Most [LaCE](#) and [Contributor Dashboard](#) issues have a frontend component. Focus on issues that aren’t marked as “backlog”. For UI demonstrations, you can tackle a non-backlog responsiveness issue from the [LaCE team](#). You might also find [this tutorial](#) helpful.

⁵ To develop this skill: Solve part of issue [#21646](#) by covering the CUIs for one or more sets of users.

[jobs](#), with tests.⁶

(Optional) Oppia Community References

Who you worked with	Context in which you worked with them
Nitish Kumar	Collaborated on learner dashboard acceptance tests and worked together on debugging and resolving multiple learner-facing issues.
Hardik Goyal	Worked together on removing Object Factories from parts of the codebase and collaborated on writing the TDD for the Flashback feature, including design and implementation discussions.
Yash Bahuguna	Helped with setting up the local development environment and getting the development server running successfully.
Meeth Gangwar	Worked closely on debugging issues and discussing possible fixes during development.

Section 3: Proposal Details

Note: This section is largely adapted from [Oppia TDD Template \(Dec 2024\) -- PLEASE MAKE A COPY](#). **We strongly recommend reading that doc**, since it has detailed explanations of how to fill out the various sections. It also contains links to sample TDDs, so that you can see what others have done in the past.

Problem Statement

The Topic Page is the main entry point for learners to understand a topic, start lessons, resume progress, and practice skills on Oppia. However, the current Topic Page does not clearly guide learners on what to do next.

Learners find it difficult to:

1. Understand what the topic is about when visiting for the first time.
2. Identify which lesson or practice to start, resume, or review.
3. See clear progress status for lessons and skills.

⁶ To develop this skill: Solve issues that are labelled with "[Beam jobs](#)". For information about how these jobs work, please see this [wiki page](#). Jobs should be submitted for testing by following [these instructions](#). Note that, in some cases, you might need to run an audit job to get information about what exactly the issues are on the server before you can create a fix-up job. You might also find [this tutorial](#) helpful.

4. Understand lesson availability in their selected website language.
5. Navigate smoothly when returning for another study session.

Because of these issues, learners may feel confused, take extra time to decide their next step, or stop learning altogether.

This project aims to redesign the Topic Page by adding clear visual cues, progress indicators, and state-based actions, so that learners can easily understand their learning status and confidently move forward.

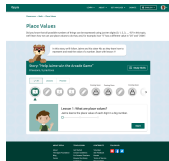
Target Audience	- Learners
Core User Need	- As a learner, I want to clearly understand what a topic is about so that I can decide if it is right for me. - As a learner, I want to easily see my lesson and skill progress so that I know what to continue next. - As a learner, I want clear guidance for starting, resuming, reviewing, or practicing content without confusion. - As a learner using a non-English language, I want to know lesson availability in advance and choose another language if needed.

Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that ..."</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements .
1	Understanding Topic Page	As a learner who has visited the topic page for the first time, I need to understand what that topic is about, so that I can decide if that topic is right for me.	Visit the Topic Page	Mocks / PRD
			Read introduction about the topic	
			Decide if the topic is right for them	
2	Starting First	As a learner who has	Visit the Topic Page	Mocks / PRD

	Lesson	already decided which Math topic I want to study, I need to check the lessons that Oppia offers for that topic, so that I can choose a lesson and start learning/improving my skills in Math.	See list of available chapters/lessons Choose a chapter See a clear CTA to start the lesson Start a lesson	
3	Lesson unavailability in the website language	As a learner who has chosen a language other than English for the website, I want to know if the lesson's not available in my language so that I don't waste time doing it.	Visit oppia.org Select preferred language (e.g. Brazilian Portuguese or PT-BR) Visit Topic Page See the list of chapters/lessons See that the lesson is not available in the preferred language (e.g. PT-BR)	Mocks / PRD
4	Secondary language selection	As a learner who has identified that a lesson is not available in my preferred language and is familiar with another language, I need to be able to choose another language for that lesson, so that I can start learning in a language that I understand.	- Visit Topic Page - See the list of chapters/lessons - See that the lesson is not available in the preferred language (e.g. PT-BR) - Select a secondary language (e.g. Spanish) - Start the lesson in the secondary language - Visit Topic Page - See the list of chapters/lessons - Click to start lesson - View dialog box - Select a secondary language (e.g. Spanish) - Start the lesson in the secondary language	Mocks / PRD
5	See status of previously started lessons	As a logged-in learner who has previously started a lesson, but hasn't finished it, and now is returning to the topic page, I want to see the status of progress, so that I can identify which lesson is in-progress and resume it.	Visit Topic Page See the list of chapters/lessons See status of progress bar of each chapter/lesson	Mocks / PRD
6	Resuming a lesson	As a logged-in learner who has identified an	See uncompleted progress bar	Mocks / PRD

		uncompleted lesson in the topic page, I want to resume that lesson, so that I can complete it.	Click on "Resume"	
7	Viewing and reviewing completed lesson(Out of scope of this project)	As a logged-in learner who has identified a completed lesson in the topic page, I want to review the story together with the correct answers, so that I can remember what I've learned in the lesson.	See completed progress bar	Mocks / PRD
			Click on CTA to view the lesson	
			See dialog asking to confirm if they would like to start over the lesson or review	
			Confirm that want to review the lesson	
			Review the lesson/story with correct answers	
8	Starting over a previously completed lesson	As a logged-in learner who has completed a lesson a long time ago and forgot everything about it, I want to retake the lesson from the beginning, so that I can be reintroduced to the Math concepts.	Visit Topic Page	Mocks / PRD
			See the list of chapters/lessons	
			See completed progress bar	
			Click on CTA to view the lesson	
			See dialog asking to confirm if they would like to start over the lesson or review	
			Confirm that want to start over the lesson	
			Start over the lesson	
9	Representation of Chapter Availability	As a learner, I want to identify available chapters, so that I'm aware how far I can progress in learning this topic currently and how much more content can I anticipate in the future	View list of 'coming soon' chapters in Story Card	PRD 
			Attempt all available chapters	
			Optionally, subscribes to email notifications for future chapter launches	
10	Review in another study session	As a learner who has completed one or more lessons previously, and now is returning to the	Visit Topic Page	Mocks / PRD
			See the list of chapters/lessons	

		platform for another study session, I want to review the previously learned skill, so that I remember what I've learned previously.	See skills related to the lessons	
			Click on CTA study the skill	
11	Practice in another study session	As a learner who has completed one or more lessons previously, and now is returning to the platform for another study session, I want to practice the previously learned skill, so that I can solidify this knowledge.	Visit Topic Page	Mocks / PRD
			See the list of chapters/lessons	
			See skills related to the lessons	
			See option to practice	
			Click on CTA to practice the skill	
12	Seeing concepts within a skill	As a learner who wants to start a practice session, I need to know more details of the skill, so that I am aware of what I'm learning.	Visit Topic Page	Mocks / PRD
			See list of skills	
			See CTA to know more	
			See list of concepts within a skill	
13	View only lessons	As a learner who doesn't want to practice skills, I need to view the list of lessons separated from the skills, so that I can find the lessons more easily.	Visit Topic Page	Mocks / PRD
			Uncheck the checkbox for practice cards	
			View the list of lesson cards	
14	View skill progress	As a logged-in learner who is doing practice sessions, I need to view my progress in the skill, so that I am aware if I'm learning that content or not.	Visit Learner Dashboard	Mocks / PRD
			See skill mastery	
			Or	
			Visit Topic Page	
			View skill	
			View skill mastery	

Testing Plan

Critical user journeys

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
CUJ 1 – Understanding Topic Page & Starting First Lesson				
Logged-in Learner	As a learner, I want to view the topic page and see the list of lessons and practices, so that I can start learning confidently	Base Setup (applies to all CUJs unless otherwise specified): 1) Log in as a Curriculum Admin. 2) Create a Classroom titled “Math”. 3) Create a Topic titled “Addition and Subtraction”. 4) Add:1 Story, 8 Lessons (Chapters), 3 Practices, Study Guide enabled. 5) Publish the Story and Topic. 6) Enable the New Topic Page Feature Flag. 7) Log in as a learner (user@hello.com). Setup ends with a logged-in learner on the Topic Page.	Part A – Default Topic Page View (First-time user)	Part A – Expectations
			1) Open the Topic Page.	Topic Page loads successfully.
			2) Stay on the page without interaction.	The page remains in default state.
			3) Check the selected tab.	“All” tab is selected by default.
			4) Identify default lesson selection.	Lesson 1 is selected by default (no progress state).
			5) Verify lessons & practice count.	8 Lessons and 3 Practices displayed.
			6) Review progress indicator strips.	11 task icons displayed in the correct lesson → practice order.
			Part B – Progress Indicator & Scroll Behavior (Web)	Part B – Expectations (Web)
			7) Review initial scroll controls.	Right arrow visible, left arrow hidden.
			8) Click the right arrow.	Strip scrolls forward and the left arrow appears.
			9) Scroll to the end of the strip.	Right arrow hidden at end.
			10) Scroll back to the beginning.	Left arrow hidden at start.
			Part C – Starting First Lesson	Part C – Expectations
			13) Click “Start” on Lesson 1.	Lesson Player opens successfully.
CUJ 2 – Filtering Content (Lessons & Practice Tabs)				

Logged-in Learner	As a learner, I want to filter between lessons and practices on the Topic Page, so that I can focus on the type of content I want to work on.	Same setup as CUJ 1. Setup ends with a logged-in learner on the Topic Page.	Part A – Viewing Only Lessons	Part A – Expectations (Lessons Tab)
			1) Click on the “Lessons” tab.	“Lessons” tab becomes selected (active state visible).
			2) Review progress indicator row.	Only lesson icons are visible in the progress indicator. Practice icons are hidden.
			3) Review the lesson cards section.	Only lesson cards are displayed. No practice cards are shown.
			Part B – Viewing Only Practice	Part B – Expectations (Practice Tab)
			4) Click on the “Practice” tab.	“Practice” tab becomes selected (active state visible).
			5) Review progress indicator row.	Only practice icons are visible in the progress indicator. Lesson icons are hidden.
			6) Review the practice card section.	Only practice cards are displayed. No lesson cards are shown.
CUJ 3 – Lesson Unavailable in Preferred Language & Secondary Language Selection				
Logged-in Learner	As a learner who has selected a preferred website language, I want to know when a lesson is not available in that language and be able to select a secondary language so that I can still start and complete the lesson.	Base setup: Same as CUJ 1. Additional setup for this CUJ: 1) Ensure at least one lesson is NOT translated into Portuguese (or chosen language). 2) Log in as a learner (user@hello.com). 3) Change website 's preferred language to Portuguese. Setup ends with a	Part A – Detect Lesson Unavailability (Card State)	Part A – Expectations
			1) Open Topic Page.	Topic Page loads in Portuguese (preferred language).
			2) Identify a lesson that is NOT available in Portuguese.	The lesson card displays the unavailable-language state.
			3) Review lesson card controls.	Text dropdown, Audio dropdown, and Info (i) icon are visible.
			4) Identify a lesson that is translated.	No language dropdowns or Info icon displayed.
			5) Attempt to click “Start” without selecting a secondary language.	Users are prompted to select a text language before starting.

		logged-in learner on Topic Page (Portuguese UI).	Part B – Info (i) Tooltip Behavior	Part B – Expectations
			6) Click the Info (i) icon.	Tooltip appears in Portuguese explaining lessons unavailable in the selected language.
			7) Close tooltips using “Got it” or “X” ..	Tooltip closes successfully.
			8) Click outside tooltip.	Tooltip closes automatically when clicking outside
			Part C – Secondary Language Selection via Dropdown (Card Level)	Part C – Expectations
			9) Open Text dropdown.	Available languages displayed.
			10) Select English as a secondary language.	English appears selected.
			11) (Optional) Select audio language.	Audio selection works but is optional.
			12) Click “Start”.	Lesson opens in selected secondary text language.
			13) Confirm overall UI language.	Website UI remains in Portuguese.
			Part D – Dialog Flow (User Clicks Start Without Selecting Text Language)	Part D – Expectations
			14) Refresh Topic Page.	Topic Page reloads in Portuguese and no secondary language is pre-selected for the untranslated lesson..
			15) Click “Start” without selecting the text language.	A dialog appears requiring the learner to select a text language before starting.
			16) Review dialog structure.	Dialog content is displayed in Portuguese.
			17) Observe dialog structure.	Dialog contains mandatory Text dropdown, optional Audio dropdown, Cancel button, disabled “Start lesson” button.
			18) Select lesson language in dialog.	“Start lesson” button becomes enabled.

			19) Click Cancel.	Dialog closes; learner remains on Topic Page.
			20) Click Start → select text language → click “Start lesson”.	The lesson opens in the selected secondary language.
			Part E – Per-Lesson Language Persistence & Resume	Part E – Expectations
			21) Start lessons in English.	The lesson begins in English text.
			22) Exit lesson midway.	Progress is saved.
			23) Return to Topic Page.	Previously selected English text remains pre-selected in the dropdown for that lesson.
			24) Click Resume.	Lesson resumes in the selected secondary language (English).
			25) Open a different untranslated lesson.	No secondary language is pre-selected; learners must choose again.
			26) Review overall language behavior.	Secondary language selection is saved per lesson only and does not change the global website language.

CUJ 4 – In-Progress Lesson (See Status, Resume, Start Over)

Logged-in Learner	As a logged-in learner who has previously started a lesson but hasn't finished it, I want to see its progress on the topic page and be able to resume or start over.	Base setup: Same as CUJ 1. Additional setup for this CUJ: 1) Log in as a learner (user@hello.com). 2) Start Lesson 3. 3) Exit before completing the lesson. Setup ends with a logged-in learner on the Topic Page with Lesson	Part A – In-Progress Status Visible on Topic Page	Part A Expectations
			1) Open Topic Page.	Lesson 3 appears marked as in progress.
			2) Look at Lesson 3 in the progress indicator row.	Lesson 3 icon shows in-progress visual state (not default, not completed).
			3) Look at the progress bar on the Lesson 3 card.	Progress bar reflects partial completion (not 0%, not 100%).
			4) Look at the CTA button on the Lesson 3 card.	CTA displays “Resume”.
			5) Look at other lessons.	Other lessons that were not started show default state with “Start” CTA.

		3 in-progress.	Part B – Resume Lesson	Part B Expectations
			6) Click “Resume” on Lesson 3.	The lesson opens from the last saved checkpoint.
			7) Continue the lesson briefly and then leave.	Progress remains saved.
			Part C – Start Over Uncompleted Lesson	Part C Expectations
			8) Click “Resume” again on Lesson 3.	A dialog appears informing the learner that the lesson is already in progress.
			9) Choose “Resume”.	Lesson resumes from the last saved checkpoint.
			10) Click “Resume” again and choose “Start over”.	Lesson restarts from the first card.
			11) Return to Topic Page after starting over.	Progress resets to initial state (no partial completion).
CUJ 5 – Status of Previously Started Lessons (Logged-out User)				
Logged-out User	As a logged-out learner, I should not see saved progress since it cannot be associated with my account.	Base setup: Same as CUJ 1. Additional setup for this CUJ: 1) Log in as a learner (user@hello.com). 2) Start Lesson 2 and exit before completing. 3) Log out. Setup ends with a logged-out user on the Topic Page after previously having partial progress.	1) Open Topic Page while logged out.	Lessons appear in their default state.
			2) Go to the lesson that was previously started (Lesson 2).	The lesson does not show any in-progress indication.
			3) Look at the lesson card.	No partial progress bar is shown.
			4) Look at the CTA button on that lesson card.	CTA displays “Start” (not “Resume”).
			5) Refresh the page.	No progress appears after refresh.
CUJ 6 – Practice in Another Study Session (Completed Practice)				

Logged-in Learner	As a learner who has completed one or more lessons previously, and is returning to the platform for another study session, I want to practice the previously learned skill, so that I remember what I've learned previously.	Base setup: Same as CUJ 1. Additional setup for this CUJ: 1) Log in as a learner (user@hello.com). 2) Complete Lesson 1. 3) Complete Practice 1 fully. Setup ends with a logged-in learner on the Topic Page with Practice 1 completed.	Part A – View Completed Practice State	Part A Expectations
			1) Open Topic Page.	Practice 1 appears marked as completed.
			2) Click the “Practice” tab (if not already selected).	Practice 1 is visible with completed state.
			3) Look at the Practice 1 card.	Progress shows 100%.
			4) Look at the CTA button.	CTA displays “Practice”.
			Part B – Start Practice Again	Part B Expectations
			5) Click “Practice” on completed Practice 1.	Practice session starts immediately.
			6) Complete or exit practice.	The system treats it as a new attempt.
			Part C – Optional Study Before Practice	Part C Expectations
			7) Click “Study” on Practice 1.	Learner is redirected to related lesson content.
8) Return to the Topic Page.	Topic Page loads with practice state unchanged.			
CUJ 7 – View Skill Progress (Logged-in Learner)				
Logged-in Learner	As a logged-in learner doing practice sessions, I want to see my skill mastery progress so that I know whether I have not started, am in progress, or have mastered the skill.	Base setup: Same as CUJ 1. Additional setup for this CUJ: 1) Log in as a learner (user@hello.com). 2) Complete Lesson 1 (so Practice 1 is unlocked). Setup ends with a logged-in learner on the Topic Page.	Part A – Skill Not Started	Part A Expectations
			1) Open Topic Page.	Practice 1 appears available.
			2) Go to the Practice tab (if not already selected).	Practice 1 card is visible..
			3) Look at the progress bar.	Progress bar shows 0% (default state).
			4) Check the progress indicator row.	The practice icon shows the default state.
			5) Check the CTA button.	CTA displays “Practice”.
			Part B – Skill In Progress	Part B Expectations
			6) Click “Practice” on Practice 1 and attempt some questions.	Partial mastery is saved.
			7) Exit the practice session and return to the Topic	Topic Page loads successfully.

			Page.	
			8) Look at the progress bar.	Progress bar reflects partial mastery (not 0%, not 100%).
			9) Check the progress indicator row.	Practice icon shows in-progress state.
			Part C – Skill Mastered	Part C Expectations
			10) Complete Practice 1 fully.	Skill mastery reaches 100%.
			11) Return to Topic Page.	Topic Page loads successfully.
			12) Look at the progress bar	Progress bar shows 100%.
			13) Check the progress indicator row.	The practice icon shows the completed/mastered state with a checkmark.
			14) Check the CTA button.	CTA displays “Practice” (new session allowed).

CUJ 8 – View Skill Progress (Logged-out User)

Logged-out User	As a logged-out learner, I should not see saved skill progress since it cannot be associated with my account.	Base setup: Same as CUJ 1. Additional setup for this CUJ: 1) Log in as a learner (user@hello.com). 2) Complete Lesson 1. 3) Complete Practice 1 fully (reach 100%). 4) Log out. Setup ends with a logged-out user on the Topic Page.	1) Open Topic Page while logged out.	Topic Page loads successfully.
			2) Go to the Practice tab.	Practice 1 card is visible.
			3) Look at the progress bar.	No progress bar is shown.
			4) Check the progress indicator row.	The practice icon shows the default state.
			5) Check the CTA button.	CTA displays “Practice”.
			6) Refresh the page.	No progress appears after refresh.

CUJ 9 – Seeing Concepts Within a Skill (Study Skills → Concept View)

Logged-in Learner	As a learner who wants to understand a	Base setup: Same as CUJ 1.	Part A – Navigate to Study Skills	Part A Expectations
-------------------	--	-----------------------------------	--	----------------------------

	skill before starting practice, I want to see the concepts within that skill, so that I clearly know what I am learning.	Additional setup for this CUJ: 1) Create at least one Skill (e.g., "Place Names and Their Values"). 2) Add explanatory content to the Skill (concept explanation, examples, visuals). 3) Link the Skill to relevant Lessons/Practices. 4) Publish Topic, Story, and Skills. 5) Log in as a learner (user@hello.com). Setup ends with a logged-in learner on the Topic Page.	1) Open Topic Page.	Topic Page loads successfully.
			2) Click the "Study Skills" button.	The learner is redirected to the Study Skills page.
			Part B – View Available Skills	Part B Expectations
			3) View the list of skill cards.	All published skills linked to the topic are visible.
			4) Check skill titles.	Skill names display correctly.
			5) Check skill visuals/icons.	Skill visuals render properly.
			Part C – View Concepts Within a Skill	Part C Expectations
			6) Click a skill card.	Skill detail page opens.
			7) Scroll through concept explanation.	Concept content is readable and structured properly.
			8) View examples and visuals.	Examples and visuals display correctly.
			9) Click "Back to Study Skills".	Learner returns to the Study Skills page.

CUJ 10 – Serial Chapter Launch ("Coming Soon") Tag with New Topic Page

Logged-in Learner	As a learner, I want to clearly see which lessons are currently locked and marked as "Coming Soon", so that I understand what content is not yet available to me.	Base setup: Same as CUJ 1. Additional setup for this CUJ: 1) Enable the Serial Chapter Launch feature flag. 2) Create a Topic with multiple lessons (e.g., 6 lessons). 3) Configure Serial Launch settings so that: Lesson 1, 2, 3 →	Part A – Verify Unlocked Lessons (Lesson 1, 2, 3)	Part A Expectations
			1) Open the Topic Page.	Topic Page loads successfully.
			2) View the lesson strips in the progress indicator row.	All lessons appear in correct order.
			3) Click Lesson 1.	Lesson 1 opens normally.
			4) Return to Topic Page.	Return to Topic Page.
			5) Check Lesson 2 and 3.	Both appear unlocked and clickable.
			Part B – Verify Locked "Coming Soon" Lessons (Lesson 4, 5, 6)	Part B Expectations
			6) View Lessons 4–6 in the lesson strip.	Each lesson icon: Appears greyed out, displays a lock icon, shows a "Coming

		Unlocked		Soon” label above the icon and is not clickable.
		Lesson 4, 5, 6 → Locked	7) Click Lesson 4.	The lesson expands but does not navigate.
		4) Publish Topic and Story.	8) View Lesson 4 card details.	→ The title shows “Coming Soon”. → Message displayed: “This lesson will be unlocked soon.” → Progress bar is not shown. → Start button is visible but disabled.
		5) Log in as a learner (user@hello.com).	9) Attempt to click Start.	No navigation occurs.
		Setup ends with a logged-in learner on the Topic Page with Serial Chapter Launch enabled.		

CUJ 11 – “New” Badge Functionality for Newly Available Lessons

Logged-in Learner	As a learner, I want to experience the new Topic Page layout with updated lesson strip, tags, and Study Skills access, so that I can navigate the topic clearly and efficiently.	Base setup: Same as CUJ 1.	Part A – Verify “New” Badge Visibility	Part A Expectations
		Additional setup for this CUJ: 1) Enable the Serial Chapter Launch feature flag. 2) Create a Topic with multiple lessons (e.g., 6 lessons). 3) Publish Lessons 1–3 (fully available to learners). 4) Later, add additional lessons (Lesson 4, 5, 6). 5) Configure Serial Launch so that: Lessons 1–3 → Already published and accessible Lessons 4–6 → Newly launched	1) Open the Topic Page.	Topic Page loads successfully.
			2) View lesson strips (progress indicator row).	Lessons 4–6 display green “New” badge above lesson circles.
			3) Expand Lesson 4 cards.	→ Lesson title displays “New” badge next to lesson name. → The progress bar is visible. → Start button is enabled.
			4) Check Lessons 1–3.	No “New” badge displayed.
			Part B – First Interaction with a “New” Lesson	Part B Expectations
			5) Click on Lesson 4 (with “New” badge).	Lesson 4 opens normally.
			6) Exit or complete the lesson.	Return to Topic Page.
			7) Check Lesson 4 again.	“New” badge is no longer displayed.
			Part C – Behavior Across Sessions	Part C Expectations

		6) Publish updates.	8) Log out.	Session ends.
		7) Log in as a learner (user@hello.com).	9) Log back in as the same learner.	Learner returns to Topic Page.
		Setup ends with a logged-in learner on the Topic Page with newly launched lessons.	10) Check Lessons 5–6 (not yet opened).	“New” badge remains visible.
			11) Check Lesson 4 (already accessed).	“New” badge does not reappear.

Reference Used: [Mocks](#)

Mocks for serial chapter launch functionality integrating with topic page:

1) New Topic Page – Serial Chapter Launch with “Coming Soon” Indicators



[LEARN](#)
[ABOUT](#)
[GET INVOLVED](#)
[DONATE](#)


[ENGLISH](#)



[Classrooms](#) > [Math](#) > [Place Values](#)

Place Values

Did you know that all possible numbers of things can be expressed using just ten digits (0, 1, 2, 3, ... , 9)? In this topic, we'll learn how we can use place values to do that, and, for example, how "5" has a different value in "25" and "2506".



In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

Story: "Help Jaime win the Arcade Game"

8 lessons, 5 practices


[Study Skills](#)

[✓ All](#)
[Lessons](#)
[Practice](#)













Coming Soon

Coming Soon

Coming Soon



Lesson 1: What are place values?

Jaime learns the place value of each digit in a big number.



[Start](#)

ABOUT OPPIA

[About](#)
[The Oppia Foundation](#)
[Blog](#)
[Forum](#)

TEACH/LEARN

[Get Started](#)
[Creator Guidelines](#)
[For Parents/Teachers](#)
[Browse the Library](#)
[Contact us](#)

CONTRIBUTE

[Volunteer](#)
[Site Feedback](#)
[Donate](#)
[Terms of Service](#)
[Privacy Policy](#)

FOLLOW US:





2) New Topic Page – Serial Chapter Launch with “Coming Soon” Indicator in expanded view

Since the lesson is not released yet, the Start button is disabled, the progress bar is removed, and a “Coming Soon” label is shown next to the lesson title along with a short message below stating that the lesson will be unlocked soon.

[LEARN](#)
[ABOUT](#)
[GET INVOLVED](#)
[DONATE](#)

ENGLISH

[Classrooms](#) > [Math](#) > [Place Values](#)

Place Values

Did you know that all possible numbers of things can be expressed using just ten digits (0, 1, 2, 3, ..., 9)? In this topic, we'll learn how we can use place values to do that, and, for example, how "5" has a different value in "25" and "2506".

In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

Story: "Help Jaime win the Arcade Game"

8 lessons, 5 practices

Study Skills

All
 Lessons
 Practice

1

2

3

4

5

6

Coming Soon

Coming Soon

Lesson4: What are place values?

Jaime learns the place value of each digit in a big number.

This lesson will be unlocked soon.

Start

ABOUT OPPIA

[About](#)
[The Oppia Foundation](#)
[Blog](#)
[Forum](#)

TEACH/LEARN

[Get Started](#)
[Creator Guidelines](#)
[For Parents/Teachers](#)
[Browse the Library](#)
[Contact us](#)

CONTRIBUTE

[Volunteer](#)
[Site Feedback](#)
[Donate](#)
[Terms of Service](#)
[Privacy Policy](#)

FOLLOW US:

3) New Topic Page – “New” Tag for Recently Released Chapters

[LEARN](#)
[ABOUT](#)
[GET INVOLVED](#)
[DONATE](#)

ENGLISH

[Classrooms](#) > [Math](#) > [Place Values](#)

Place Values

Did you know that all possible numbers of things can be expressed using just ten digits (0, 1, 2, 3, ..., 9)? In this topic, we'll learn how we can use place values to do that, and, for example, how "5" has a different value in "25" and "2506".

In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

Story: "Help Jaime win the Arcade Game"

8 lessons, 5 practices

Study Skills

All
 Lessons
 Practice

1

2

3

New

4

New

5

New

6

Lesson4: What are place values?

Jaime learns the place value of each digit in a big number.

Start

ABOUT OPPIA

[About](#)
[The Oppia Foundation](#)
[Blog](#)
[Forum](#)

TEACH/LEARN

[Get Started](#)
[Creator Guidelines](#)
[For Parents/Teachers](#)
[Browse the Library](#)
[Contact us](#)

CONTRIBUTE

[Volunteer](#)
[Site Feedback](#)
[Donate](#)
[Terms of Service](#)
[Privacy Policy](#)

FOLLOW US:

4) Mock showing the state when a user completes all previous lessons and practices and clicks on Lesson 4 – the “New” badge is dismissed and a triangular progress pointer is displayed.

[LEARN](#)
[ABOUT](#)
[GET INVOLVED](#)
[DONATE](#)

ENGLISH

[Classrooms](#) > [Math](#) > [Place Values](#)

Place Values

Did you know that all possible numbers of things can be expressed using just ten digits (0, 1, 2, 3, ..., 9)? In this topic, we'll learn how we can use place values to do that, and, for example, how "5" has a different value in "25" and "2506".

In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

Story: "Help Jaime win the Arcade Game"

8 lessons, 5 practices

Study Skills

All
 Lessons
 Practice

1

2

3

4

5

6

Lesson4: What are place values?

Jaime learns the place value of each digit in a big number.

Start

ABOUT OPPIA

[About](#)
[The Oppia Foundation](#)
[Blog](#)
[Forum](#)

TEACH/LEARN

[Get Started](#)
[Creator Guidelines](#)
[For Parents/Teachers](#)
[Browse the Library](#)
[Contact us](#)

CONTRIBUTE

[Volunteer](#)
[Site Feedback](#)
[Donate](#)
[Terms of Service](#)
[Privacy Policy](#)

FOLLOW US:

Updated mock -

[LEARN](#)
[ABOUT](#)
[GET INVOLVED](#)
[DONATE](#)

ENGLISH

[Classrooms](#) > [Math](#) > [Place Values](#)

Place Values

Did you know that all possible numbers of things can be expressed using just ten digits (0, 1, 2, 3, ..., 9)? In this topic, we'll learn how we can use place values to do that, and, for example, how "5" has a different value in "25" and "2506".

In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

Story: "Help Jaime win the Arcade Game"

8 lessons, 5 practices

Study Skills

✓ All

Lessons

Practice

1

2

3

4

5

6

New

Lesson 4: What are place values?

Jaime learns the place value of each digit in a big number.

Start

[ABOUT OPPIA](#)

[About](#)
[The Oppia Foundation](#)
[Blog](#)
[Forum](#)

[TEACH/LEARN](#)

[Get Started](#)
[Creator Guidelines](#)
[For Parents/Teachers](#)
[Browse the Library](#)
[Contact us](#)

[CONTRIBUTE](#)

[Volunteer](#)
[Site Feedback](#)
[Donate](#)
[Terms of Service](#)
[Privacy Policy](#)

[FOLLOW US:](#)

Technical Requirements

Additions/Changes to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	New / Existing	Description of the request/response contract (and, if applicable, how it's different from the previous one). For new endpoints, include which access control decorator (e.g. "can_play_exploration", "open_access", etc.) will be used.
1.	N/A			

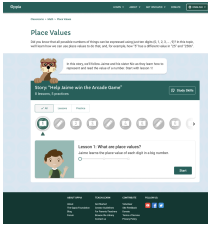
New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

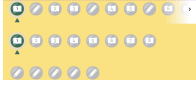
#	Endpoint URL	Request type (GET, POST, etc.)	Description of why the new call is needed, or why the changes to an existing call are needed
1.	N/A		

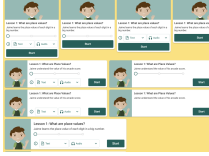
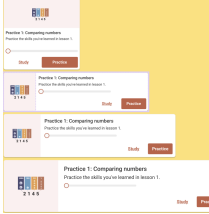
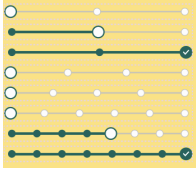
Web Server Storage Model Additions/Changes

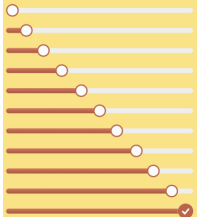
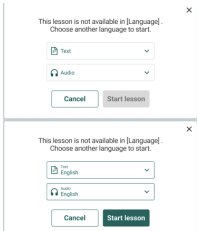
#	Datastore model	Description of changes	How will existing data be handled?
1.	N/A		

UI Screens/Components

#	ID	Existing or New?	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	TopicViewerPage Component(Entry shell)	Existing	The existing TopicViewerPageComponent remains the only entry shell. The redesigned layout will be implemented in the same component and gated behind the feature flag. It will coordinate data loading and render header, content tabs, progress strip, lesson/practice cards, and modals.	Yes	<u>Mocks</u> 	Semantic page landmarks and heading hierarchy; keyboard-accessible controls; visible focus states; screen-reader friendly section labels.
2.	TopicHeaderComponent	New	Displays the top section of the Topic Page, including breadcrumb navigation, topic title (h1), and topic description. This section provides overall context for the topic and is rendered only once at the top of the page.	Yes	<u>Mocks</u> 	Semantic heading hierarchy (h1 for topic title); breadcrumb navigation accessible via screen readers; proper focus order and visible focus states.

3.	TopicStorySectionComponent	New	Renders each story as an independent section below the topic header. For each story, display the story header (story title, lesson/practice counts, Study Skills CTA), story description, and all lessons/practices for that story. Sections are stacked vertically for multiple stories.	Yes	<u>Mocks</u> 	Story title uses appropriate heading (h2 or h3); CTA as semantic button/link; counts as readable text; focus order and visible focus states.
4.	TopicContentFilterComponent	New	Segmented filter tabs allowing learners to switch between All, Lessons, and Practice sections dynamically.	Yes	<u>Mocks</u> 	Proper tab semantics (role="tablist", role="tab"), keyboard navigation with arrow keys, active tab announced to screen readers.
5.	TopicProgressComponent	New	Horizontal progress strip container for ordered lesson/practice nodes. Composes TopicProgressCircleComponent for each node, supports active/completed/in-progress/locked/new states, and includes horizontal scroll controls for overflow content. Emits node selection and scroll actions to parent.	Yes	<u>Mocks</u> 	All nodes are keyboard operable through child buttons; locked nodes marked appropriately; left/right scroll controls are keyboard-operable buttons with descriptive aria-labels; visible focus states; semantic grouping/label for the progress strip.
6.	TopicProgressCircleComponent	New	Renders one progress node (lesson or practice) inside the progress strip. Handles visual state variants (active, completed, in-progress, locked, new), icon selection, and node-level accessibility semantics. Used only as a child of	Yes	<u>Mocks</u> 	Node rendered as keyboard-accessible button; descriptive aria-label per node; locked nodes exposed with aria-disabled/disabled

			TopicProgressComponent.			bled state; active node exposed with aria-current="step" ; visible focus ring and keyboard activation via Enter/Space.
7.	TopicLessonCardComponent	New	Topic-page lesson card displaying thumbnail, title, description, progress bar, optional language selection affordance, and CTA (Start, Resume, Review, Disabled). Supports serial-launch states (Coming Soon, New).	Yes	<u>Mocks</u> 	CTA keyboard accessible; progress exposed via ARIA attributes; locked/disabled states announced clearly; status not conveyed by color alone.
8.	TopicPracticeCardComponent	New	Topic-page practice card with related lesson context, mastery progress bar, and actions (Study, Practice). Supports repeat attempts and logged-out behaviour where mastery progress is hidden.	Yes	<u>Mocks</u> 	Buttons keyboard operable; mastery progress conveyed via ARIA progress semantics; accessible labels for actions and outcomes.
9.	CheckpointBarComponent	Existing	Renders lesson progress for topic cards using lesson states (not_started, in_progress, completed, coming_soon), node markers, and fill progression aligned to lesson flow	Yes	<u>Mocks</u> 	Uses role="progressbar" with aria-valuenow, aria-valuemin, aria-valuemax; each node has descriptive aria-label and tooltip; keyboard focus on interactive nodes; status is conveyed by text/state, not color alone.
10.	TopicPracticeProgressBarComponent	New	Renders practice mastery progress for topic cards using practice-specific states and mastery percentage. Supports	Yes	<u>Mocks</u>	Uses role="progressbar" and readable mastery text for screen readers;

			repeat attempts and mastered-state rendering			interactive points/buttons are keyboard operable with aria-labels; completion state is announced semantically; no color-only communication.
11.	LanguageSelect orComponent	New	This component is not only a simple dropdown. It handles the reusable language selection UI and logic when a lesson is not available in the learner's preferred language. The component includes: 1)Text language dropdown 2)Optional audio language dropdown 3)Info tooltip explaining language availability 4)Validation to ensure a text language is selected before starting the lesson This component will be reused in two places shown in the mocks: 1)Inside the lesson card when the lesson is not available in the preferred language. 2)Inside the LanguageSelectionModalComponent, which appears when the learner clicks "Start" without selecting a language first.	Yes	<u>Mocks</u> 	Fully keyboard navigable dropdowns; explicit labels and helper text; validation messaging associated with fields; required/error state announced.
12.	LanguageSelect orModalComponent	New	Modal dialog displayed when a learner attempts to start a lesson without selecting required text language. Hosts the LanguageSelectionComponent and starts/cancels actions.	Yes	<u>Mocks</u> 	Accessible dialog semantics (role="dialog", aria-modal); focus trap; initial focus and return focus handling; Escape key support.

13.	NewProgressReminderModalComponent	Existing	Reusable modal for in-progress/completed lessons. Shows checkpoint progress and provides Resume or Start Over actions based on lesson state. Already used in lesson player; adapted for topic page with configurable button labels.	Yes	<u>Mocks</u> 	Uses accessible dialog semantics (role="dialog", aria-modal), ensures all actions are fully keyboard operable, provides clear and descriptive action labels, manages focus safely when opening and closing.
-----	-----------------------------------	----------	---	-----	--	---

Data Handling and Privacy

#	Type of data	Description	Why do we need to store this data?	Anonymized?	Can the user opt out?	Wipeout policy	Takeout policy
1.	N/A						

Connection to Existing Work

1) PR #20406 – Feature Flag: The redesigned Topic Page will be gated behind the existing feature flag. The old Topic Page remains unchanged when the flag is OFF, ensuring safe rollout and easy rollback.

2) Serial Chapter Launch (GSoC 2023): Existing lesson locking, “Coming Soon” states, and “New” badge logic will be reused. Only the UI layer will change; backend release logic remains unchanged.

3) New Lesson Player: Existing progress calculation, resume logic, and language selection functionality will be reused without API modifications.

4) Learner Dashboard (Progress Tab): Existing progress visualization logic will inform linear and circular progress indicators on the Topic Page.

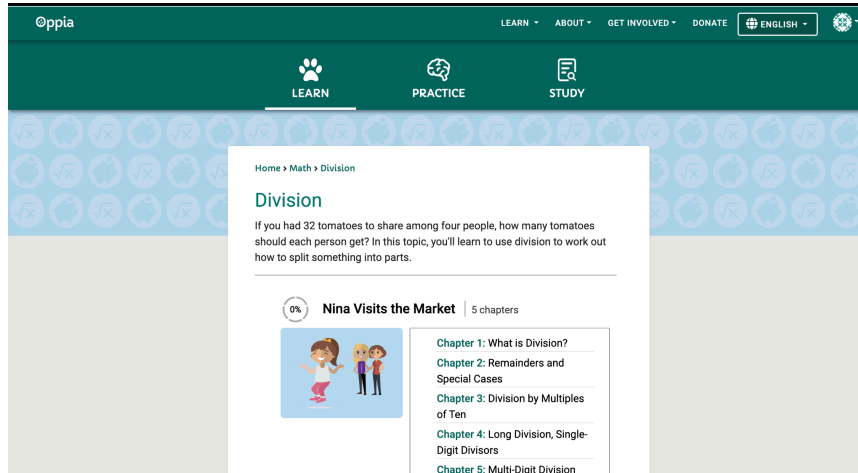
5) Topic Editor Page: Card layout patterns and styling conventions will follow the current design system for visual consistency.

Other Requirements

N/A

Section 3.2: HOW

Existing Status Quo



The current Topic Page is stable and primarily story-centric. Learners navigate content through story tiles in the Learn tab, while Practice and Study are presented in separate sections. Although this structure is technically sound, it fragments the learner journey across multiple tabs instead of presenting a single, continuous learning path. As a result, learners can view stories and chapters, but do not get a unified sequence that combines lessons, practice, and progress in one place.

Current Technical Architecture and Flow

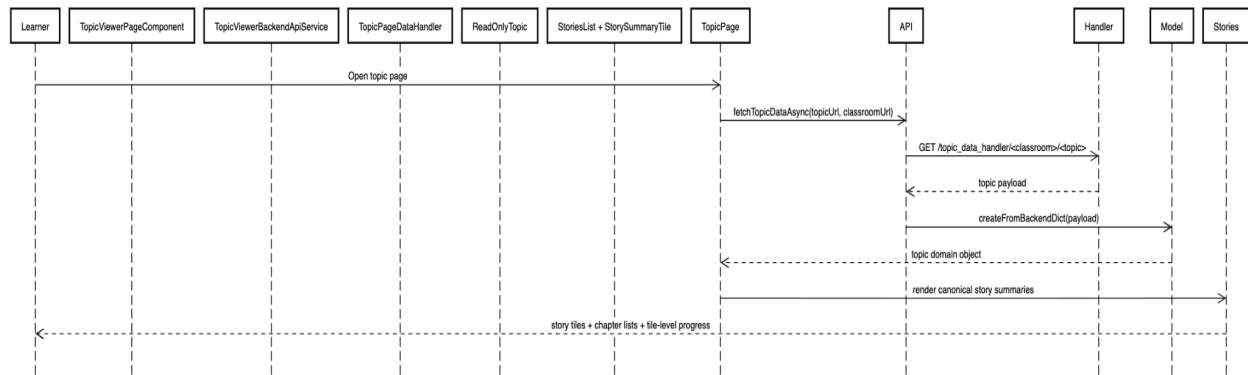
When a learner opens a topic page, the flow is as follows:

- The learner navigates to a topic URL
- The frontend loads [TopicViewerPageComponent](#).
- The component fetches data using:
[TopicViewerBackendApiService.fetchTopicDataAsync\(...\)](#)
- The request is sent to:
[/topic_data_handler/<classroom_url_fragment>/<topic_url_fragment>](#)
- The backend handler [TopicPageDataHandler](#) processes the request
- The response is parsed into the [ReadOnlyTopic](#) model on the frontend

The UI is then rendered using:

[core/templates/pages/topic-viewer-page/topic-viewer-page.component.html](#)

- Stories are displayed using the reusable [oppia-story-summary-tile component](#)
- Each tile shows chapter lists, progress indicators, and navigation links to lessons



Current Rendering and Data Usage

- Topic data is fetched via [fetchTopicDataAsync\(\)](#).
- Story tiles are rendered using [oppia-story-summary-tile](#)
- Learner progress is derived from story and chapter completion data (e.g., [completed_node_titles](#)) along with learner progress services
- [ContentTranslationLanguageService](#) is part of the lesson player and is not directly used in topic page rendering
- Backend provides [canonical_story_dicts](#) with the UI primarily prioritizing canonical stories

Backend Capabilities Already Available

The backend and existing models already provide strong support for a redesigned experience:

1. Canonical story dictionaries
2. Node-level metadata (Published, Ready To Publish, Draft)
3. Story and chapter completion data
4. Skill mastery mapping ([degrees_of_mastery](#))
5. Practice tab visibility controls
6. Feature flag infrastructure (topic redesign, serial chapter launch)

Because of these capabilities, the redesign primarily requires frontend composition and interaction changes, rather than backend modifications.

Current Limitation

While the system is technically robust and modular, the experience is optimized for content presentation rather than learner guidance. Lessons, practice, and progress are not presented as a unified journey, making it harder for learners to understand what to do next.

This gap between content structure and guided learning flow is the key problem the redesign aims to solve.

Current Status of Feature Flag

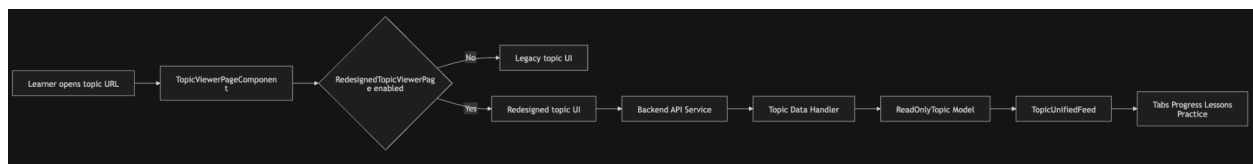
The feature flag `redesigned_topic_viewer_page` is defined in both:

- Backend: `feature_flag_list.py`
- Frontend: `feature-status-summary.model.ts`
- It is currently part of `DEV_FEATURES_LIST`
- Available only in development environments
- Not enabled for test or production environments

The feature flag is correctly implemented and wired, and the redesigned Topic Page can be toggled in development mode.

Solution Overview

The redesign introduces a learner-guided topic experience while preserving the existing backend contract. Instead of merging all content into a single stream, the UI organizes content story-wise, where each story is rendered as an independent section. This approach keeps the data model simple, avoids unnecessary transformation, and ensures that each story maintains its own learning flow, progress, and actions.

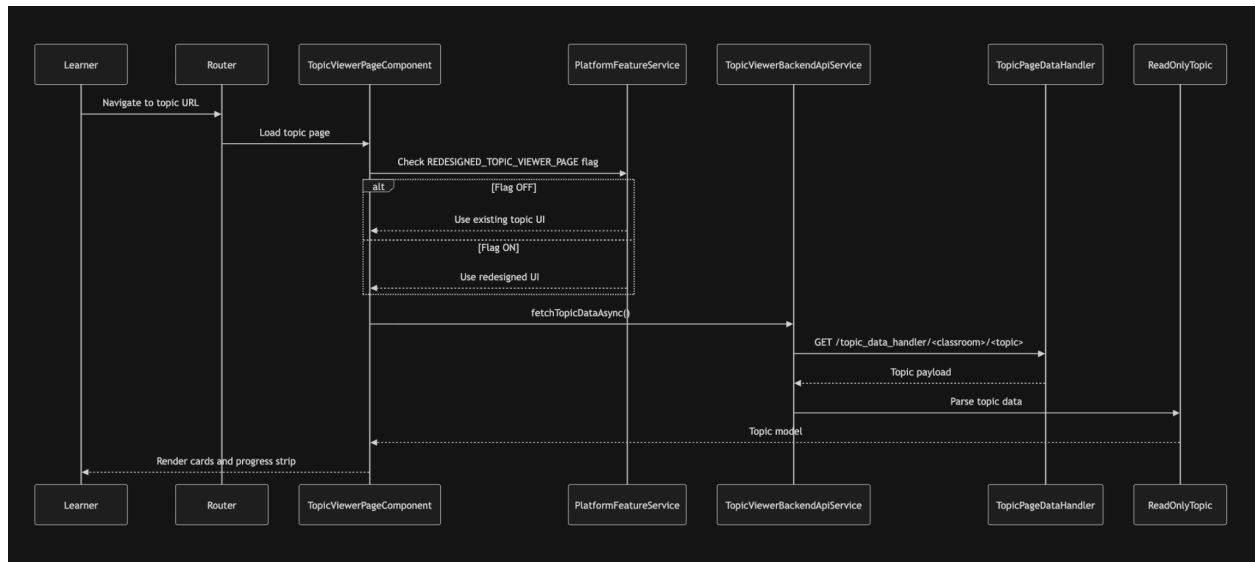


At a high level:

- 1) Keep using [/topic_data_handler/<classroom_url_fragment>/<topic_url_fragment>](#).
- 2) Keep using `ReadOnlyTopic` as the parsed domain model.
- 3) Render each story as a separate vertical section containing its story header, progress strip, lesson cards, and practice cards.
- 4) Ensure lessons and practices are ordered strictly based on backend story structure, without merging across stories.
- 5) Maintain story-level independence for progress, CTA states (Start / Resume / Review), and sequencing.
- 6) Reuse existing patterns such as serial chapter launch, language/audio preferences, and mastery tracking.
- 7) Gate all redesigned UI behind feature flags to allow safe rollout, testing, and easy rollback.

Sequence Diagram (Data Flow)

The following sequence diagram illustrates how data flows through the system when a learner opens a topic page.



Support for multiple stories

The Topic Page will support multiple stories by rendering each story as an independent section within a vertically scrollable layout. Instead of merging all lessons and practices into a single stream, each story is displayed separately, preserving its structure, progress, and learning flow.

1. Fetching Topic Data

The existing data fetching flow remains unchanged. Topic data is fetched using the existing topic API and parsed into the **ReadOnlyTopic** model, which already provides canonical stories.

2. Preparing Story Data (Frontend)

After fetching, the **TopicViewerPageComponent** prepares the story data for rendering.

3. Rendering Multiple Stories

The page iterates over the list of stories and renders each story as a separate section.

For each story, the UI includes:

1. Story header (title, lesson/practice counts, CTA)
2. Progress strip (scoped to that story only)
3. Lesson cards (derived from chapters)
4. Practice cards (mapped to that story)

Story sections are stacked vertically, creating a scrollable page layout.

4. Story-Level Independence

Each story maintains its own state and behavior:

1. Lesson sequence follows the story's chapter structure
2. Progress is calculated per story
3. CTA states (Start / Resume / Review) are story-specific

There is no mixing of:

1. Lessons across stories
2. Practice progress across stories
3. Progress indicators across stories

This ensures correctness and avoids confusion for the learner.

5. User Experience

From the learner's perspective:

1. The page is vertically scrollable
2. Multiple stories are visible in a single view
3. Learners can:
 - a. Continue progress within a story
 - b. Switch between stories easily
 - c. Understand progress per story clearly

This structure maintains clarity while still allowing smooth navigation across stories.

Example UI (multiple stories layout):

Place Values

Did you know that all possible numbers of things can be expressed using just ten digits (0, 1, 2, 3, ..., 9)? In this topic, we'll learn how we can use place values to do that, and, for example, how "5" has a different value in "25" and "2506".



In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

Story: "Help Jaime win the Arcade Game"

8 lessons, 5 practices

Study Skills

✓ All Lessons Practice



Lesson 1: What are place values?

Jaime learns the place value of each digit in a big number.



Start



In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

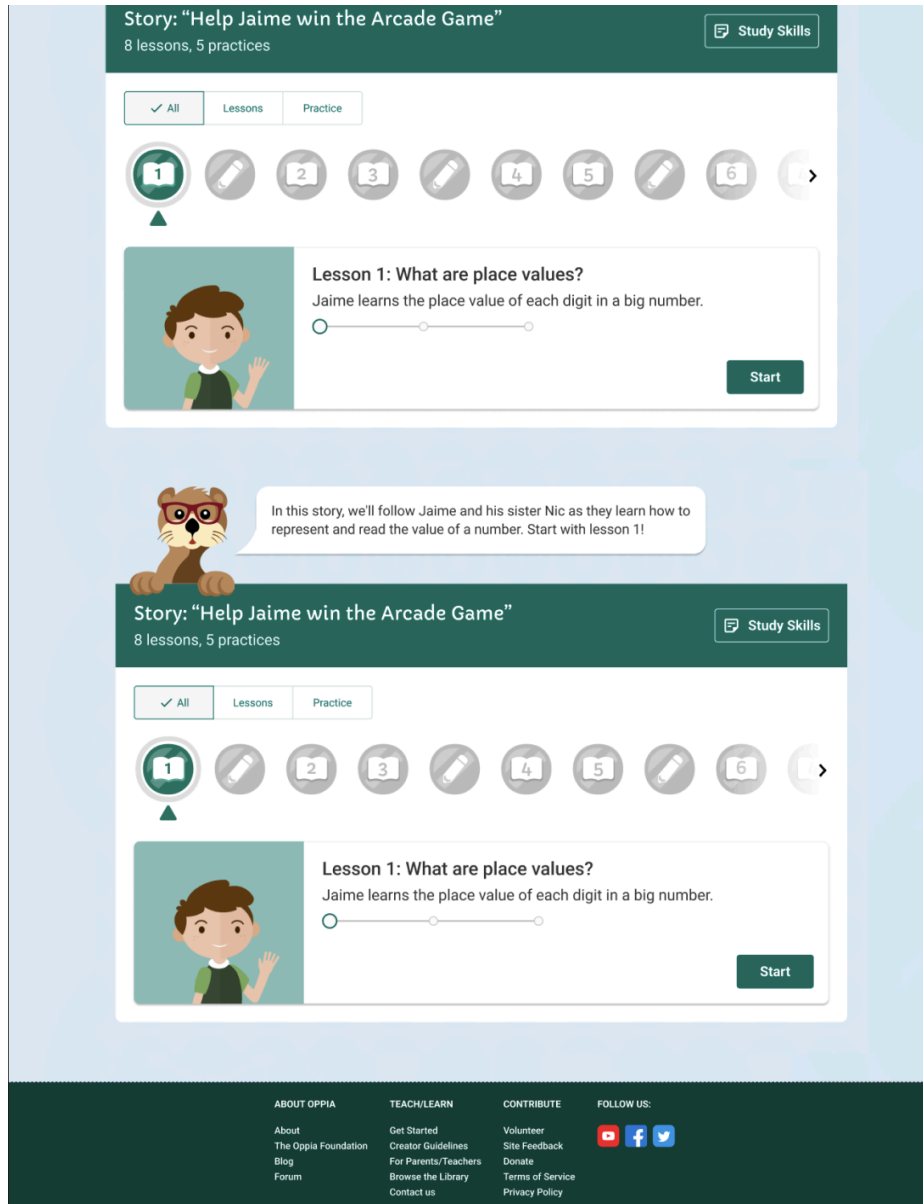
Story: "Help Jaime win the Arcade Game"

8 lessons, 5 practices

Study Skills

✓ All Lessons Practice





This mock illustrates how multiple stories are stacked vertically, each with its own header, progress, and lesson/practice cards, while allowing the learner to scroll through all stories in a single page.

6. Backend Context (No Changes Required)

The backend already supports multiple stories through [canonical_story_summaries](#). No backend changes are required; the redesign builds entirely on top of the existing contract.

Edge Cases Handling

The redesigned Topic Page explicitly handles key edge cases related to stories, lessons, and practice availability. The UI adapts dynamically based on the data returned from the backend, while preserving

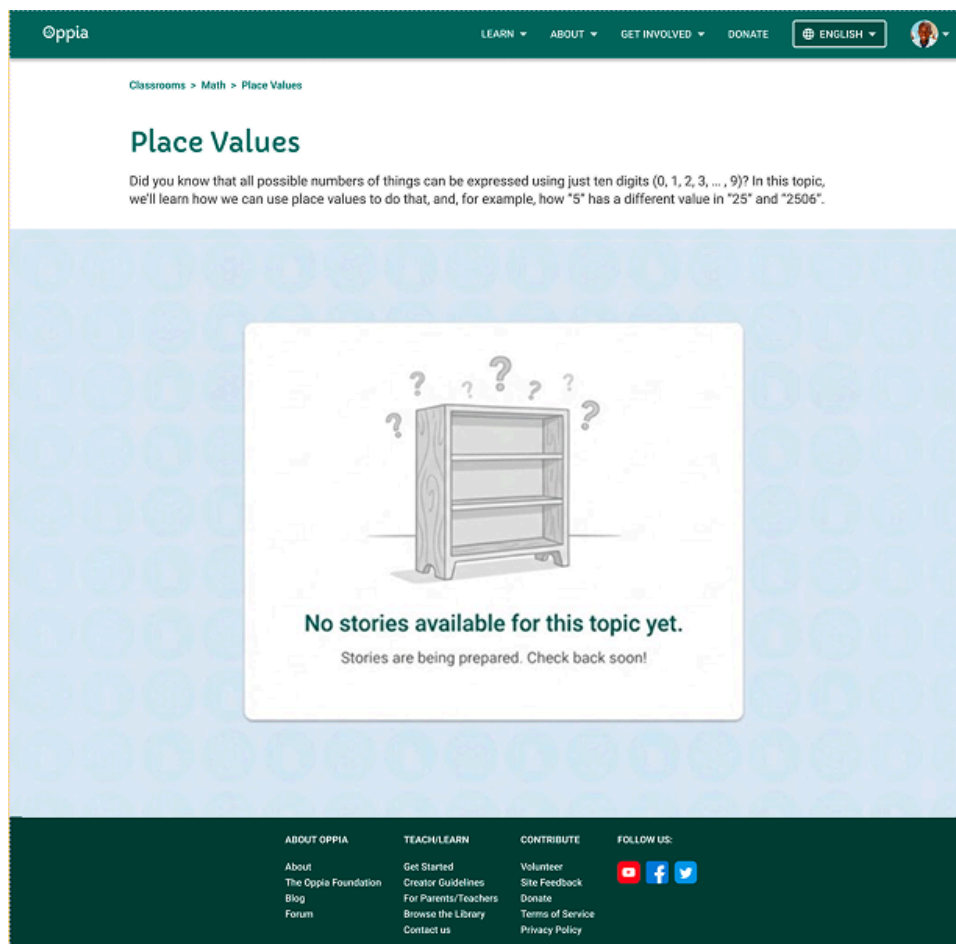
layout consistency and avoiding empty or broken states. All edge cases reuse existing layout structures to ensure visual consistency and avoid layout shifts.

1. No Stories in a Topic

When a topic contains no stories, the page does not attempt to render any story sections. Instead, the UI presents a centered empty state within the content area.

- The topic header (title and description) is still displayed as usual.
- The main content area shows an empty state card with a message such as: *"No stories available for this topic yet."*
- A subtle illustration is used to make the state informative rather than abrupt.

This ensures that the page remains visually complete and communicates clearly to the learner that content is not yet available, rather than appearing broken.

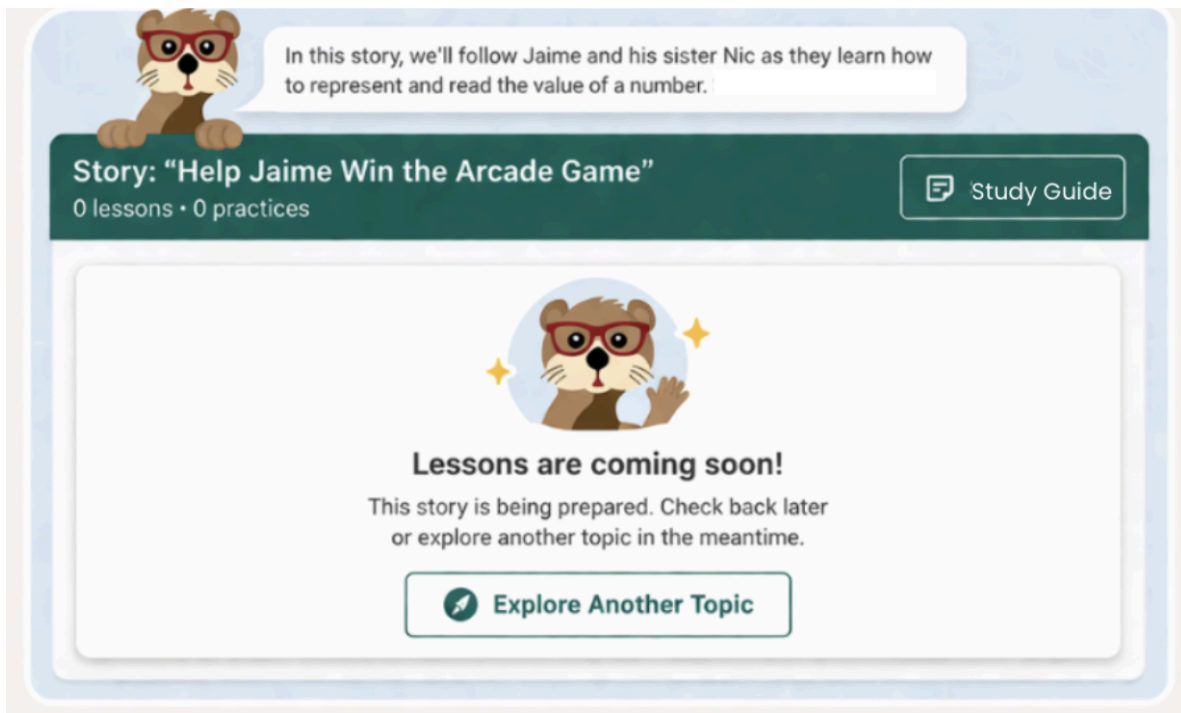


2. Story with No Lessons

If a story exists but does not contain any lessons:

- The story header (title, counts, CTA like “Study Guide”) is still rendered.
- Instead of showing tabs or a progress strip, the main content area displays a dedicated empty state card.
- The empty state clearly communicates that content is not yet available, for example: “Lessons are coming soon! This story is being prepared. Check back later or explore another topic in the meantime.”
- A primary CTA such as “Explore Another Topic” should take the learner to the classroom homepage.

This approach maintains structural consistency while clearly indicating that content is coming soon.



3. Only Practice Available (No Lessons)

When a story contains only practice sessions:

- The tab component adapts to show only relevant options:
 - “Practice” (active)
- The “Lessons” tab is hidden or disabled.
- The progress strip reflects only practice nodes.
- Practice cards are rendered as the primary content.

This avoids showing empty lesson states and keeps the interface focused on available content.

Oppia
LEARN
ABOUT
GET INVOLVED
DONATE
ENGLISH

Classrooms > Math > Place Values

Place Values

Did you know that all possible numbers of things can be expressed using just ten digits (0, 1, 2, 3, ..., 9)? In this topic, we'll learn how we can use place values to do that, and, for example, how "5" has a different value in "25" and "2506".

In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

Story: "Help Jaime win the Arcade Game"

0 lessons, 8 practices

[Study Skills](#)

[Practice](#)

Practice 1: Comparing numbers

Practice the skills you've learned in lesson 1.

Study
Practice

ABOUT OPPIA

- About
- The Oppia Foundation
- Blog
- Forum

TEACH/LEARN

- Get Started
- Creator Guidelines
- For Parents/Teachers
- Browse the Library
- Contact us

CONTRIBUTE

- Volunteer
- Site Feedback
- Donate
- Terms of Service
- Privacy Policy

FOLLOW US:

4. Only Lessons Available (No Practice)

When only lessons are available:

- The tab component adapts accordingly:
 - "Lessons" (active)
- The "Practice" tab is hidden or disabled.
- The progress strip shows only lesson nodes.
- Lesson cards are rendered normally.

This ensures the UI remains clean and avoids redundant or empty tabs.

Oppia

LEARN ▾ ABOUT ▾ GET INVOLVED ▾ DONATE

ENGLISH ▾

Classrooms > Math > Place Values

Place Values

Did you know that all possible numbers of things can be expressed using just ten digits (0, 1, 2, 3, ..., 9)? In this topic, we'll learn how we can use place values to do that, and, for example, how "5" has a different value in "25" and "2505".

In this story, we'll follow Jaime and his sister Nic as they learn how to represent and read the value of a number. Start with lesson 1!

Story: "Help Jaime win the Arcade Game"

8 lessons, 0 practices

Study Skills

✓ Lessons

1 2 3 4 5 6 7 8

Lesson 1: What are place values?

Jaime learns the place value of each digit in a big number.

0%

Start

ABOUT OPPIA
About
The Oppia Foundation
Blog
Forum

TEACH/LEARN
Get Started
Creator Guidelines
For Parents/Teachers
Browse the Library
Contact us

CONTRIBUTE
Volunteer
Site Feedback
Donate
Terms of Service
Privacy Policy

FOLLOW US:
YouTube Facebook Twitter

5. Mixed Availability Across Stories

When a topic contains multiple stories with different availability states (e.g., one story has lessons, another has only practice, another has no lessons):

- Each story independently adapts its UI based on its content
- Empty states are shown only within the affected story section
- Other stories remain unaffected and fully interactive

This ensures that content availability in one story does not impact the visibility or usability of others.

Feature Flag:

@ppia > Release coordinator
BEAM JOBS
FEATURES
MISC

redesigned_topic_viewer_page

Description:

This flag activates the redesigned topic viewer page and makes it accessible to learners.

Feature Stage:

Dev (can only be enabled on dev server).

Rollout percentage for logged-in users (0-100):

0

Force-enable for all users:

No

Last Updated:

The feature has not been updated yet.

User Groups:

Add User Group
Users in the selected user groups will always have the feature enabled, regardless of the rollout percentage. If both user groups and rollout percentage are set, the feature will be enabled for users present in the user groups and for the percentage of logged-in users specified.

Save

Clear Changes

This feature will only be enabled for the above percentage of **logged-in** users. We recommend rolling out the feature to **logged-in** users first, and once we are sure that doing so doesn't break anything, extending it to **logged-out** users as well using the "Force-enable for all users" control below.

Setting this to "Yes" will force-enable the feature for all the users, both **logged-in** and **logged-out**. Setting this property will disable the effect of all the other properties and enable the feature for all the users.

- In the redesigned flow, the page decides between the old and new UI using a feature flag. It shows the new design only when the flag is enabled.
- The old (legacy) flow is kept unchanged. This makes rollback easy, since we can switch back to the old UI just by turning off the flag. [RedesignedTopicViewerPage](#) controls the topic page layout, while [SerialChapterLaunchLearnerView](#) controls chapter availability i.e **"New"** and **"Coming Soon"** separately.
- Keeping these flags orthogonal is important: one controls presentation migration, the other controls release timing of chapter availability behavior. This follows existing feature plumbing patterns in `core/feature_flag_list.py`, frontend enum wiring in `core/templates/domain/feature-flag/feature-status-summary.model.ts`, and runtime access through [platformFeatureService.status.<Flag>.isEnabled](#).
- Once the new design is fully stable, we can remove the old code in a simple cleanup step. This just means removing the flag check and deleting the old UI code. Overall, this approach allows safe rollout, easy rollback, and simple removal of the old design.

Determining the Order of Lessons and Practice

Step 1: Use `get_ordered_nodes()` to get the ordered list of lessons (chapters) for the story.

Step 2: For each lesson, look up its associated skill IDs.

Step 3: For each skill ID, find the corresponding practice (from subtopics/skills).

Step 4: Merge or display the practices alongside or after the relevant lesson, based on the skill linkage.

1. Get Ordered Lessons (Chapters)

Backend:

- Each story contains an ordered list of nodes (chapters).

- The method `get_ordered_nodes()` in `core/domain/story_domain.py` returns the chapters in the correct order.

Frontend:

- The topic page fetches the canonical stories and their chapters from the backend and displays them in this order.
- See: `topic-viewer-page.component.ts`

2. Get Practices (Skills) Associated with Each Lesson

- Backend:
 - Each chapter (`StoryNode`) has a list of `acquired_skill_ids` (skills taught in that lesson).

Subtopics (in the topic model) also have an ordered list of skills.

 - See: `core/domain/topic_domain.py`, `get_subtopics()`, `get_skill_ids()`
- Frontend:
 - For each lesson, look up its `acquired_skill_ids`.
 - For each skill ID, find the corresponding practice (from the subtopic/skill list).
 - See: `practice-tab.component.ts` for how practices are rendered.

3) Merge Lessons and Practices

Algorithm:

- Fetch the ordered list of lessons using `get_ordered_nodes()`.
- For each lesson, fetch its `acquired_skill_ids`.
- For each skill, find the corresponding practice (from subtopics/skills).
- In UI, display the lesson, then display the related practices

```
lessons = story.get_ordered_nodes()
for lesson in lessons:
    display_lesson(lesson)
    for skill_id in lesson.acquired_skill_ids:
        display_practice_for_skill(skill_id)
```

References in Codebase

- Lessons:
 - `core/domain/story_domain.py`: `StoryContents.get_ordered_nodes()`
 - `core/templates/pages/topic-viewer-page/topic-viewer-page.component.ts`
- Practices:
 - `core/domain/story_domain.py`: `StoryNode.acquired_skill_ids`
 - `core/domain/topic_domain.py`: `get_subtopics()`, `get_skill_ids()`
 - `core/templates/pages/topic-viewer-page/practice-tab/practice-tab.component.ts`

The order of lessons is determined by `get_ordered_nodes()` in the story backend. The practices are linked to lessons via `acquired_skill_ids` and are ordered by subtopic/skill order in the topic backend. The frontend merges these by iterating over lessons and, for each, displaying the related practices.

Language and Audio Initialization

The redesigned Topic Page uses the same language and audio initialization behavior as the existing lesson player, so users get consistent translation and voiceover behavior from topic card to lesson launch.

What is reused

The Topic Page continues to rely on existing frontend services:

1. [ContentTranslationLanguageService](#)
2. [ContentTranslationManagerService](#)
3. [VoiceoverPlayerService](#)

No new backend dependency is introduced for this flow.

Language priority

For each lesson, preferred text language is resolved in this order:

1. Previously selected language for that lesson in the current session (if available)
2. Site UI language
3. Exploration default language

Important clarification on persistence

Per-lesson language and audio choices are session-level only. They are not persisted per lesson across sessions. Global preferences like site language may still persist for logged-in users.

Topic Page Language Selector Flow

On topic page load:

1. Frontend reads available text languages and audio options for each lesson from topic payload.
2. Text and audio selectors are populated per lesson.
3. Default text language is resolved using the priority above.
4. The audio state is initialized against that text language.

For audio initialization per lesson:

1. If session-level audio selection exists and is valid, keep it.
2. Else if audio exists for selected text language, preselect a valid audio option.
3. Else set audio to off and show audio as unavailable.

When user changes text language:

1. Revalidate current audio selection.
2. Keep audio if still valid.
3. If invalid, clear audio or pick a valid fallback based on UX rule.
4. Disable audio control when no valid audio exists and show helper text.

When user changes audio:

1. Save selection in session state for that lesson.
2. Keep text unchanged.
3. Apply selection to lesson-launch payload.

Lesson launch flow from Topic Page

When user clicks Start:

1. Resolve preferred text language from session, then site language, then lesson default.
2. Validate text availability for that lesson.
3. Validate audio availability for the resolved text context.
4. If text is unavailable, open language selector modal.
5. If audio is unavailable, continue in text-only mode or prompt users based on UX rules.
6. Launch lesson with resolved text language and optional audio language.

Runtime behavior in lesson player

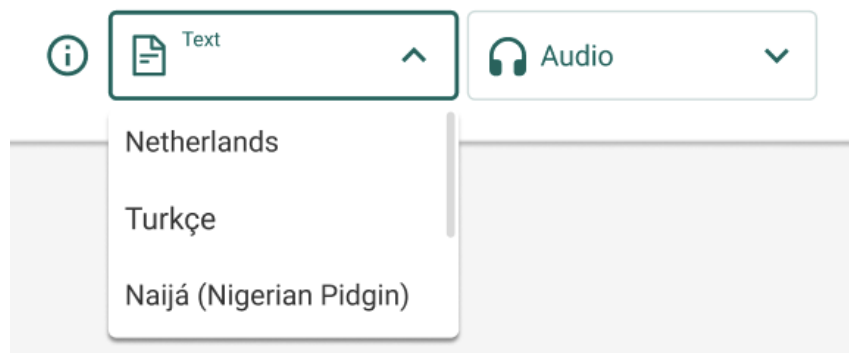
At lesson initialization:

1. Set active content language.
2. Load translations.
3. Configure voiceover if valid audio exists.
4. If audio is not available, continue gracefully in text-only mode.

If a user changes language later in the player, the same validation and reconfiguration flow is reused so text and audio remain in sync.

Backend responsibility

Backend provides language and audio metadata in payload. Frontend owns selection logic, fallback handling, session state, and launch-time validation.



For progress handling, we will reuse checkpoint semantics, not checkpoint visuals. The checkpoint logic in `checkpoint-bar.component.ts` already demonstrates the semantic transition model through methods like [getCompletedProgressBarWidth\(\)](#) and [updateLessonProgressBar\(\)](#), with states such as incomplete, in-progress, and completed. We will adopt this semantic idea when deriving topic-card progress states.

However, we will not directly reuse the checkpoint-bar component for topic cards, because it is tightly coupled to lesson-player runtime services and card-position events. We will also avoid using one single shared card-level progress component for both lesson and practice in this redesign, since these two surfaces have different progression logic and need independent rendering behavior. Also, the existing shared progress component in `progress-bar.component.ts` is already used across multiple existing flows, so extending it for topic-card-specific semantics increases regression risk.

Instead, we will implement two separate topic-page components:

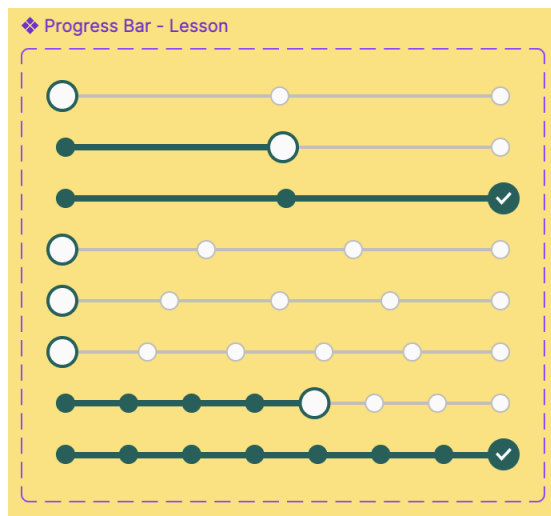
1. CheckpointBarComponent
2. TopicPracticeProgressBarComponent

Both components will render linear progress visuals at card level, but each will compute state and percentage using its own domain-specific logic.

Proposed APIs:

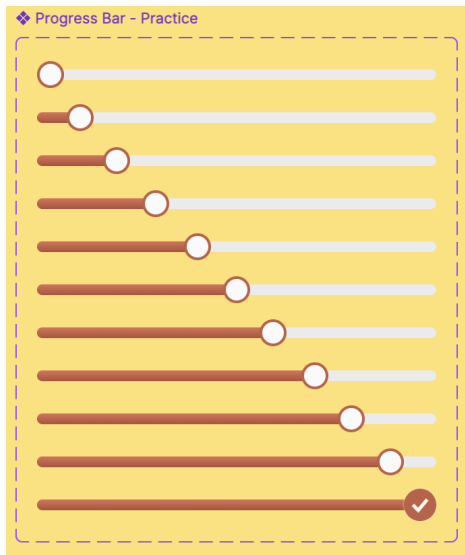
```
<oppia-checkpoint-bar
  [progressPercentage]="lessonProgressPercentage"
  [lessonState]="lessonState">
</oppia-checkpoint-bar>
```

- `progressPercentage`: number from 0 to 100, derived from lesson progression.
- `lessonState`: `not_started`, `in_progress`, `completed`, or `coming_soon`.



```
<oppia-topic-practice-progress-bar
  [progressPercentage]="practiceMasteryPercentage"
  [practiceState]="practiceState">
</oppia-topic-practice-progress-bar>
```

- progressPercentage: number from 0 to 100, derived from practice mastery.
- practiceState: not_started, in_progress, or mastered.



Scroll behavior for progress indicators:

The progress indicator displays a horizontal row of task icons (representing lessons and practices). The visibility of the left and right scroll arrows depends on how many tasks are visible at once, which varies by device and screen size.

Arrow Visibility Thresholds:

- Scroll arrows are only shown if the total number of tasks exceeds the visible limit for the current screen type:
 - Web/Tablet (landscape): 9
 - Tablet (portrait): 7
 - Mobile (landscape): 8
 - Mobile (portrait): 4
- If the number of tasks is greater than the threshold, the right arrow appears. If the user scrolls, the left arrow appears as well. If the number of tasks fits within the visible area, no arrows are shown.

Scroll Behavior:

- When a scroll arrow is clicked, the container scrolls horizontally by half the visible threshold for that screen type.
- If the threshold is an odd number, we round up (e.g., for a threshold of 9, scroll by 5).
- The scroll values for each screen type are:
 - Web/Tablet (landscape): scroll by 5
 - Tablet (portrait): scroll by 4
 - Mobile (landscape): scroll by 4
 - Mobile (portrait): scroll by 2

Arrow Visibility Updates:

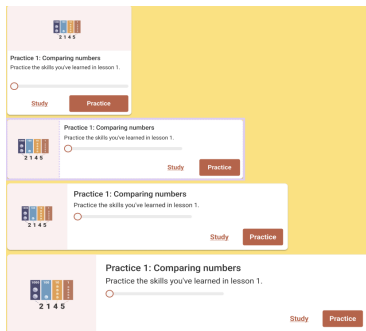
- The right arrow is shown if there are more tasks to the right, beyond the visible frame.
- The left arrow appears only after the user has scrolled away from the start.
- Arrows are hidden when the scroll reaches the start or end of the list.

This logic ensures that navigation remains intuitive and consistent across devices, and that users can always access all tasks, even on smaller screens.



For **practice cards**, skill/micro-progress should follow learner dashboard patterns. The existing mastery batching and conversion path can be seen in `core/templates/domain/learner_dashboard/learner-dashboard-backend-api.service.ts` (`fetchSubtopicMastery(...)` with `selected_topic_ids`) and progress rendering in `core/templates/pages/learner-dashboard-page/progress-tab.component.ts` + `core/templates/pages/learner-dashboard-page/skill-card/skill-card.component.ts` and `.html`.

The **redesigned topic practice cards** should reuse the same mastery-to-percent transformation and keep the circular mastery indicator purely presentational while preserving the existing mastery calculation pipeline. Color/state mapping required by mocks can be layered in the card renderer without changing mastery source semantics.



Study Guides View Implementation

In the redesigned Topic Page, Study Guides will no longer be exposed as a top-level tab inside the main topic viewer layout. Instead, they will be surfaced through a prominent “Study Skills” button within each story section, as shown in the mock. This changes the entry point to the study flow, but it does not require building an entirely new study-guide system. The goal is to make study resources feel more directly connected to the learner’s current story context, while still preserving Oppia’s existing study-guide architecture.

In the current codebase, the topic-level study listing already exists through [subtopics-list.component.ts](#) and [subtopics-list.component.html](#). This component already renders the “Study Skills for <topic>” heading and displays a collection of study guide tiles using `oppia-subtopic-summary-tile`. Structurally, this is already very close to the UI shown in the redesign mock, where learners see a responsive grid of study guide cards after clicking the “Study Skills” button.

When a learner opens a specific study guide, Oppia already has an existing study-guide viewer flow implemented in [subtopic-viewer-page.component.ts](#) and [subtopic-viewer-page.component.html](#). This page fetches the selected subtopic’s study guide data and renders the revision content. It also already supports the newer restructured study-guides layout behind the relevant feature flag, so the redesign can continue building on this existing page instead of introducing a duplicate viewer.

Because of this, the implementation should focus on reusing and restyling existing study components, rather than creating a parallel study-guides feature. The main UI change will be at the Topic Page level: the redesigned story section will include a Study Skills CTA that navigates learners into the existing study-skills listing flow. From there, learners can continue selecting a specific study guide card and proceed into the already existing subtopic viewer flow.

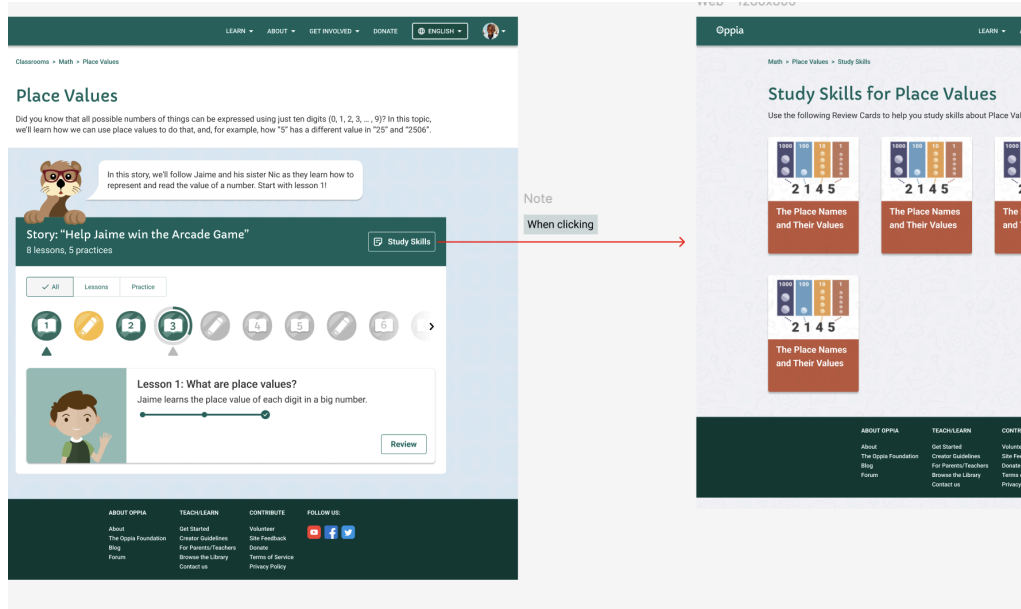
The implementation approach will be:

- Add a new “Study Skills” CTA inside each redesigned story section on the Topic Page.
- Route that CTA to the existing study-skills listing flow for the topic.
- Reuse `subtopics-list.component.ts` as the Study Guides landing/list page.
- Reuse the existing `oppia-subtopic-summary-tile` based listing from `subtopics-list.component.html`.
- Reuse `subtopic-viewer-page.component.ts` for the individual study guide.

This means the redesign does not need a brand-new study-guides page component from scratch. What it needs is a new entry point from the redesigned Topic Page. This is the safer and more maintainable approach, because it preserves existing backend flows, existing navigation behavior, and existing study-guide rendering logic.

From a user-flow perspective, the study experience will work as follows:

- The learner opens the redesigned Topic Page.
- Inside a story section, the learner clicks the “Study Skills” button.
- The learner is taken to the study-skills listing page for that topic.
- The learner sees a responsive grid/list of study guide cards.
- Clicking a card opens the existing study guide for that subtopic.



Serial chapter launch

Current Implementation:

- In the current implementation, serial launch behavior is already fully wired through Feature Flag - [SerialChapterLaunchCurriculumAdminView](#), [SerialChapterLaunchLearnerView](#) and story-tile rendering logic.
- At render time, [oppia-story-summary-tile](#) checks the learner-view serial flag in `core/templates/components/summary-tile/story-summary-tile.component.ts`, and the [template](#) in `core/templates/components/summary-tile/story-summary-tile.component.html` splits chapters into an **Available(New)** section for Published nodes and a **Coming Soon** section for Ready To Publish nodes.

Redesign Approach:

- The redesign preserves all existing chapter availability semantics and only updates the presentation. Instead of grouping chapters into separate sections, the redesigned UI displays lessons as stateful cards within a single ordered sequence.
- Published chapters become launchable cards, ready-to-publish chapters become disabled **Coming Soon** cards, and draft chapters remain hidden.
- The **"New"** label continues to follow existing eligibility rules (recently published and unvisited), and is implemented using the existing [ChapterLabelVisibilityService](#) to avoid duplicating logic.
- To align with mocks, the key change is representational rather than behavioral. The current view separates **"New"** and **"Coming Soon"** into grouped sections inside each story tile; the redesigned view should place these states inline in one deterministic lesson sequence, with clear visual states and CTA behavior.
- Launchable cards should expose **Start/Resume/Review** actions based on progress, while coming-soon cards remain clearly non-interactive, visually disabled, and accessibility-labeled as

unavailable. This keeps learner expectations stable while improving navigational clarity and reducing context switching.

Consistency with Topic Editor

To keep the redesigned Topic Page consistent with Topic Editor, I will follow the same reuse pattern that Oppia already uses in the current codebase. Instead of building a completely separate preview implementation, the editor will continue to use a lightweight preview wrapper, while the main UI is rendered through shared learner-facing components. This helps ensure that what creators see in preview stays aligned with what learners actually see on the live Topic Page.

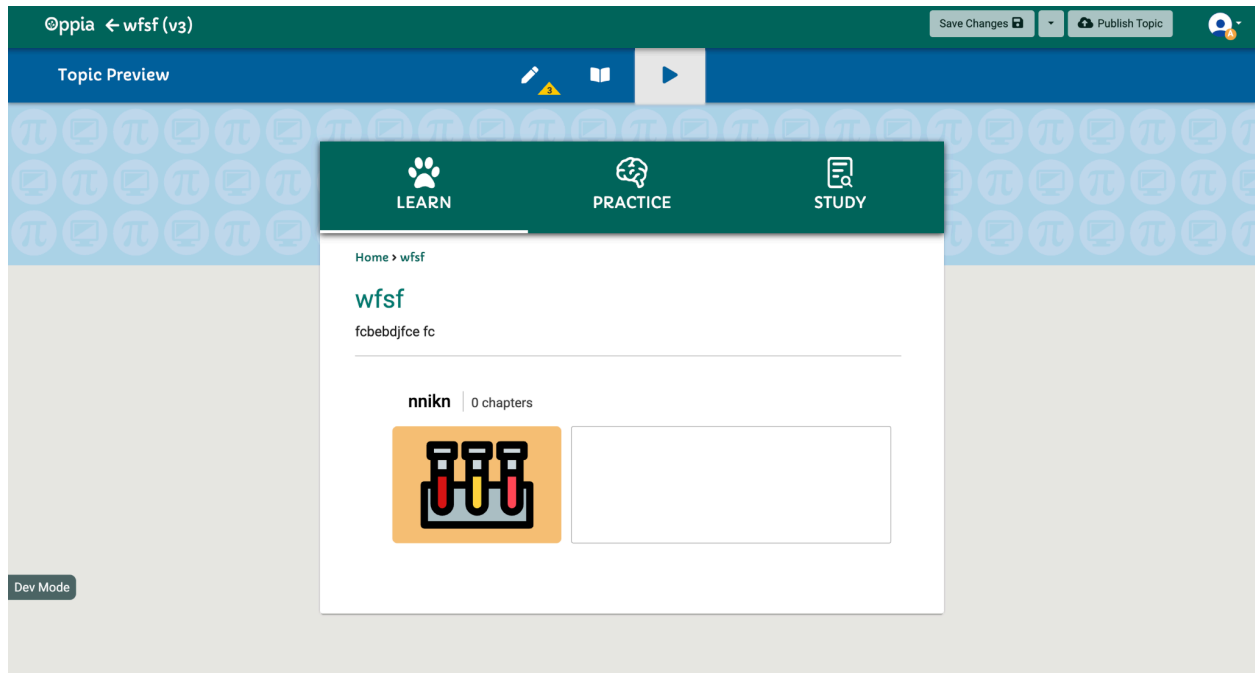
At present, Topic Editor already has a dedicated preview tab in [topic-editor-page.component.html](#), which renders [topic-preview-tab.component.ts](#). Inside this preview, Oppia already reuses learner-side topic viewer sections rather than maintaining a duplicate preview-only page. In particular, the preview template renders stories-list, subtopics-list, and practice-tab in [topic-preview-tab.component.html](#).

This reuse is also reflected at the module level. The Topic Editor module in [topic-editor-page.module.ts](#) imports [topic-viewer-player-common.module.ts](#), which already exposes shared topic-viewer components for use in both learner and preview contexts. So the current codebase already establishes the architectural pattern that the redesign should continue.

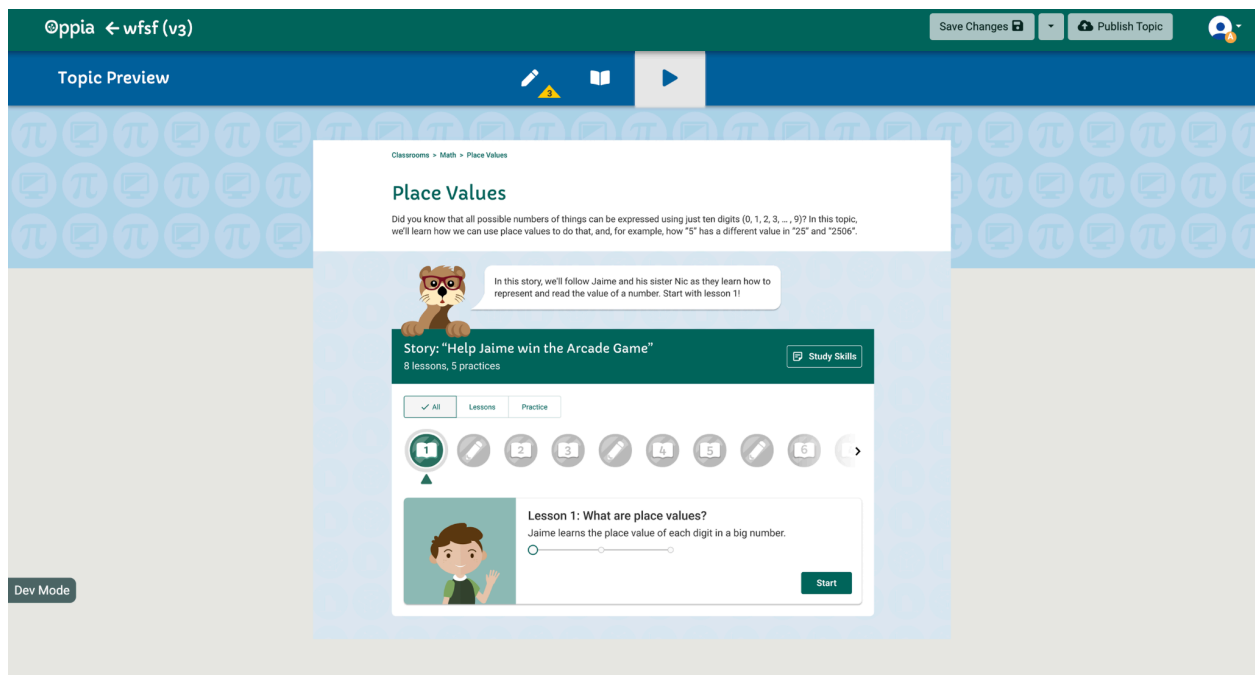
For the redesign, the learner shell will remain `topic-viewer-page.component.ts`, and the editor preview shell will remain `topic-preview-tab.component.ts`. The redesigned Topic Page UI will be implemented as reusable presentational components, and both shells will consume the same component set. Any small preview-specific differences will be handled through lightweight inputs or CSS hooks, similar to the existing preview-mode styling used in [topic-viewer-stories-list.component.css](#), [practice-tab.component.css](#), and [subtopics-list.component.css](#).

The existing preview navigation flow from the Topic Editor navbar will also be preserved. Currently, preview-related navigation is managed in [topic-editor-navbar.component.ts](#), and the redesign will continue to build on this flow rather than introducing a new preview system.

Current Implementation of the Topic Preview tab in Topic Editor page



Mock for topic editor consistency with the new topic page



Existing Implementation of “Coming Soon” and “New” Tags (Current Topic Editor / Topic Page)

Topic Editor

[Back to Topics Dashboard](#) <Addition

Details

Name

Addition

Topic name should be at most 39 characters.

Topic URL Fragment*

add

The topic URL fragment is used to uniquely access the topic viewer page. It should consist of one or more hyphen-separated words, all in lowercase, with at most 20 characters in total. Please use meaningful keywords, and avoid using words like "and", "of", or "the". This topic can be accessed at the following URL: `localhost/learn/math/add`

Description

description

Topic description should be at most 240 characters.

Page Title Fragment For Web

fragm

Canonical Stories

+ ADD STORY

Title	Status	Chapters	Chapter Publication Notifications	
dummy_titl e0	Partially Published	1 Published 1 Awaiting Publication	1 upcoming launch (next 13 days)	
dummy_titl e1	Not Published	0 Published	-	
dummy_titl e2	Not Published	0 Published	-	
dummy_titl e3	Not Published	0 Published 2 Awaiting Publication	-	

1 Subtopics

1 Skills

4 Stories

Dev Mode

©ppia < Addition (v11)

Save Changes

Unpublish Topic

Topic Preview

Home > math > Addition

Addition

description

0% dummy_title0 2 chapters

Dummy Thumbnail

Available 1 chapter

Chapter 1: dummy chapter New

1

Coming Soon 1 chapter

Chapter 2: dummy chapter 2

dummy_title1 0 chapters

Dummy Thumbnail

Available 0 chapters

Dev Mode

Determining the Next Steps (Green Arrow Position)

a. Data Model and Progress Tracking

The learner's progress is tracked at the topic, story, and lesson (chapter) level using backend endpoints such as `/story_progress_handler` and `/learnerdashboardtopicsandstoriesprogresshandler/data`.

On the frontend, the following data is used:

- `canonicalStorySummaries`: provides the ordered list of stories and lessons
- `completed_node_titles`: identifies completed lessons within each story

This ensures that both ordering and progress state are available for determining the next step.

b. Frontend Logic for Next Step

The next step is determined by selecting the first incomplete lesson in the story sequence.

```
for (let i = 0; i < storyModel.getAllNodes().length; i++) {  
  if (!storyModel.isNodeCompleted(storyModel.getAllNodes()[i].getTitle())) {  
    nextNodeIndex = i;  
    break;  
  }  
}
```

Once identified, the UI highlights this lesson using a green arrow indicator.

c. Edge Cases

1. If all lessons are completed, the green arrow is not shown
2. If the learner is new, the arrow points to the first lesson
3. If progress data is missing, the system defaults to the first incomplete lesson

Third-Party Libraries

No new third-party libraries are required.

"Service" Dependencies

This feature adds no new service dependencies.

Key High-Level and Architectural Decisions

Decision 1: Supporting Multiple Stories on the Topic Page

We considered the following alternatives:

- Keep canonical stories as separate UI sections and render them independently (**chosen**)
- Build a unified normalized feed from canonical stories and render a single merged stream

We choose rendering stories as independent sections because:

- It preserves the natural structure of stories as defined in the backend, where each story remains a clear learning unit
- It avoids unnecessary frontend complexity and reduces regression risk by eliminating the need for merging and re-ordering lessons or practices across stories
- It ensures correctness, since progress, sequencing, and CTA states remain scoped to each story
- It aligns directly with the existing Oppia data model ([ReadOnlyTopic](#), story summaries), requiring no additional transformation layer
- It improves clarity for learners by maintaining clear boundaries between stories

The above approaches are contrasted in detail in the following table:

	Alternative 1: Separate Story Sections (Chosen)	Alternative 2: Unified normalized feed
Control over lesson/practice ordering	Naturally follows backend story order	Requires custom merging and ordering logic
Consistency of UI elements	Scoped per story, simpler and predictable	Needs shared state across merged stream
Implementation complexity	Simple iteration over stories	Requires normalization + mapping layer
Risk of regression in learner flow	Low (no structural change to data)	Medium (new abstraction layer introduced)
Scalability for more stories	High, as stories are independent blocks	High, but adds complexity in merging logic
Clarity for learners	High (clear story boundaries)	Lower (stories visually merged)

Decision 2: Preferred Language Handling on Start Click

We considered the following alternatives:

- Launch the lesson immediately and rely on the lesson player to resolve language after entry
- Resolve the content language at the topic-card “Start” click and pass an explicit launch context, showing a modal only when required (**chosen**)

We choose resolving the language at Start click because:

- It ensures deterministic launch behavior and directly satisfies the CUJ requirement for handling unavailable preferred languages
- It aligns with the existing lesson player language and audio flow ([ContentTranslationLanguageService](#), [ContentTranslationManagerService](#), [VoiceoverPlayerService](#))
- It supports per-lesson language persistence without affecting the global site language

- It prevents confusing “wrong language on first load” scenarios and reduces unnecessary retries by learners

The above approaches are contrasted in detail in the following table:

	Alternative 1: Resolve only after lesson opens	Alternative 2: Resolve at Start click + conditional modal
Control over launch language	Lower. Learners may enter lessons before fallback is visible.	Higher. Language is finalized before navigation.
User experience on untranslated lessons	Can feel reactive/confusing (“why did it open in English?”).	Proactive and clear (prompt before launch when needed).
Alignment with CUJ requirements	Partial. Some requirements are handled post-entry.	Strong. Explicit pre-launch selection flow and persistence behavior.
Per-lesson language persistence	Harder to reason about from the topic page state.	Straightforward: save per lesson node in topic-card state.
Consistency with new lesson player architecture	Medium. Relies on downstream fallback only.	High. Mirrors the existing lesson-player initialization sequence and language/audio refresh pattern.
Code changes	Lower immediate change, but more behavior ambiguity.	Moderate changes in launch resolver + modal flow, but clearer contracts.
Risk of wrong-language first render	Higher.	Lower.

Risks and mitigations

Potential Risk	Mitigation
“Coming Soon” and “New” states may be interpreted differently after UI redesign.	Preserve existing serial-launch semantics exactly (Published launchable, Ready To Publish locked, Draft hidden). Reuse ChapterLabelVisibilityService logic for “New” and only change visual representation to card states.
Multiple stories with mixed states may create inconsistent UI perception	Multiple stories with mixed states may create inconsistent UI perception
Mastery concepts may be difficult for learners to understand.	Use clear copy and visual states (0%, in-progress, mastered) instead of exposing raw scoring formulas. Add concise tooltip/help text for mastery meaning.

Additional API calls per card can create N+1 performance regressions on large topics.	Use one topic fetch per page load, aggregate IDs for supplemental data, perform one batched fetch, normalize to maps, and cache page-level results for tab/filter toggles.
If a learner's preferred site language is unavailable for a lesson, they may be confused at Start click.	Enforce pre-launch language resolution flow: require text-language selection when needed, show explicit fallback message, keep audio optional, and persist selection per lesson node only.

Implementation Approach

This project implements the redesigned Topic Page as a frontend-led change that preserves the existing backend contract and reuses the current topic-viewer route, access validation, and learner progress infrastructure.

The Topic Page continues to fetch data using [topic-viewer-backend-api.service.ts](#), which calls the existing topic-data endpoint. No backend changes are required. The primary implementation change is a frontend orchestration layer that transforms the existing topic payload into the redesigned UI, including a structured header, filterable tabs, lesson cards, practice cards, progress indicators, and language-aware lesson launch flows.

Storage Model Layer Changes

N/A

Storage Model Migrations

N/A

Domain Objects

The redesigned Topic Page does not introduce any new backend domain objects. The backend continues to return the existing ReadOnlyTopic payload, parsed via `read-only-topic.model.ts`.

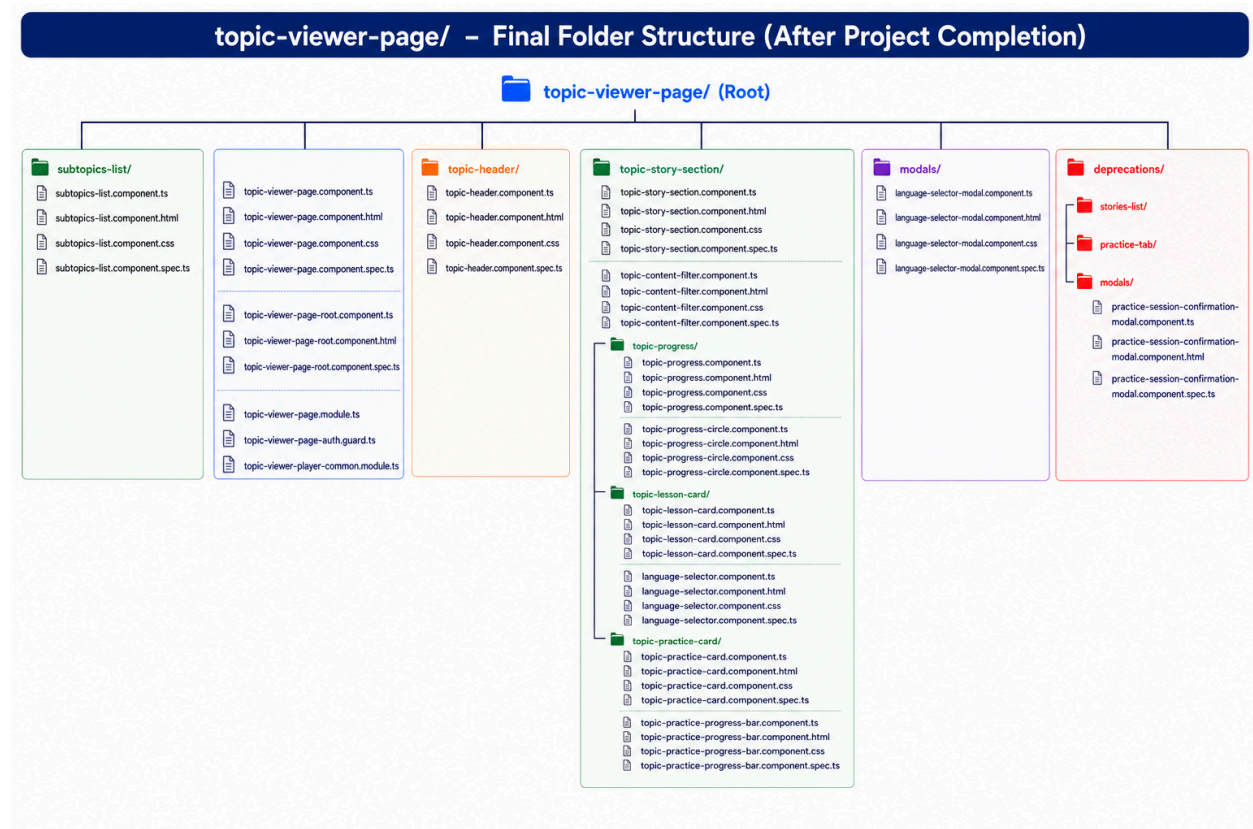
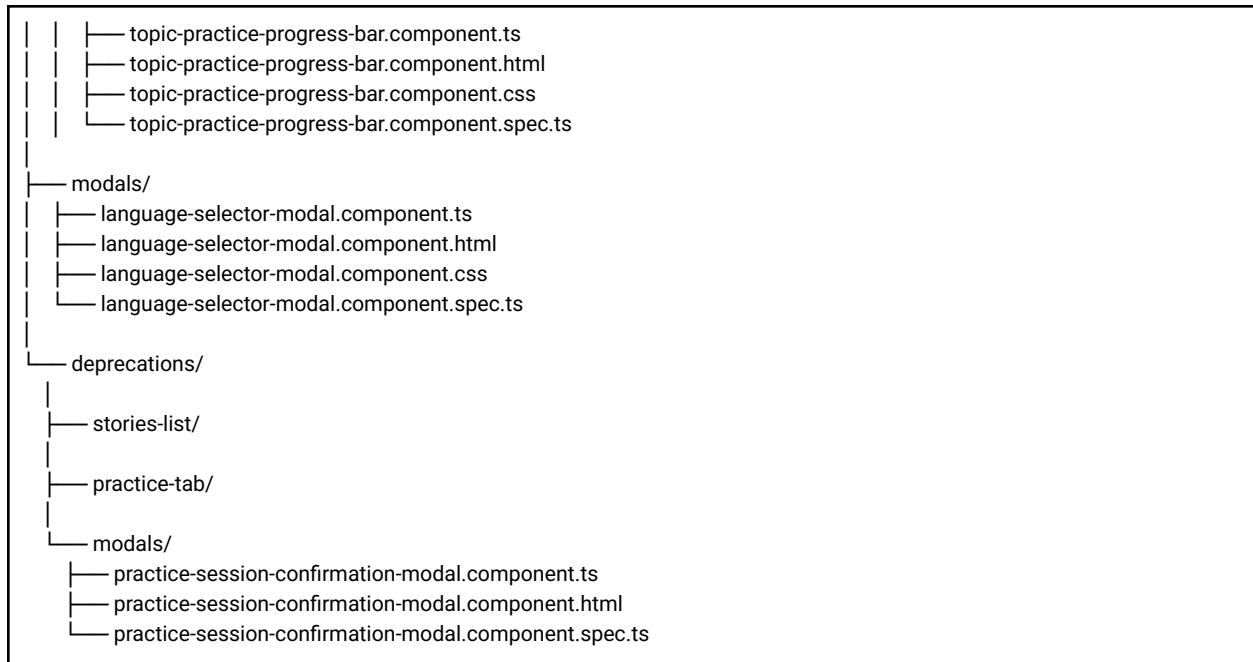
To support the redesigned UI, the frontend introduces presentation-layer view models. These are not persisted and exist only to transform backend data into a structure suitable for rendering. This keeps data fetching unchanged while enabling clean UI composition.

User Flows (Controllers and Services)

Folder Structure

```
topic-viewer-page/
├── subtopics-list/
│
└── topic-viewer-page.component.ts
```

- topic-viewer-page.component.html
- topic-viewer-page.component.css
- topic-viewer-page.component.spec.ts
- topic-viewer-page-root.component.ts
- topic-viewer-page-root.component.html
- topic-viewer-page-root.component.spec.ts
- topic-viewer-page.module.ts
- topic-viewer-page-auth.guard.ts
- topic-viewer-player-common.module.ts
- topic-header/
 - topic-header.component.ts
 - topic-header.component.html
 - topic-header.component.css
 - topic-header.component.spec.ts
- topic-story-section/
 - topic-story-section.component.ts
 - topic-story-section.component.html
 - topic-story-section.component.css
 - topic-story-section.component.spec.ts
- topic-content-filter.component.ts
- topic-content-filter.component.html
- topic-content-filter.component.css
- topic-content-filter.component.spec.ts
- topic-progress/
 - topic-progress.component.ts
 - topic-progress.component.html
 - topic-progress.component.css
 - topic-progress.component.spec.ts
 - topic-progress-circle.component.ts
 - topic-progress-circle.component.html
 - topic-progress-circle.component.css
 - topic-progress-circle.component.spec.ts
- topic-lesson-card/
 - topic-lesson-card.component.ts
 - topic-lesson-card.component.html
 - topic-lesson-card.component.css
 - topic-lesson-card.component.spec.ts
 - language-selector.component.ts
 - language-selector.component.html
 - language-selector.component.css
 - language-selector.component.spec.ts
- topic-practice-card/
 - topic-practice-card.component.ts
 - topic-practice-card.component.html
 - topic-practice-card.component.css
 - topic-practice-card.component.spec.ts



Web frontend changes

The core frontend change is that the existing topic viewer shell becomes a thin orchestration layer, while the redesigned UI is composed of focused presentational components. The page route, access guard, and backend API service remain in place; the main changes occur in how the fetched data is transformed and rendered.

Component APIs

The redesigned topic page follows a container/presentational pattern. `TopicViewerPageComponent` is the page-level container. It fetches topic data, prepares page-level and story-level view models, and passes them to child components.

`TopicStorySectionComponent` is a composite section component. It receives one prepared story-section model and passes smaller view models to its child components. It does not fetch backend data itself, but it can emit user interactions upward.

Leaf UI components such as header, filter, progress bars, lesson cards, practice cards, and language selector are presentational. They render the UI from the data passed to them and may contain small view-specific helper methods, but they do not fetch backend data or own page-level business logic.

Modal components are opened through `NgbModal` and receive data through `modalRef.componentInstance`, not through direct template bindings from `TopicViewerPageComponent`.

1) TopicHeaderComponent

API:

```
import {Component, Input} from '@angular/core';

@Component({
  selector: 'oppia-topic-header',
  templateUrl: './topic-header.component.html',
})

export class TopicHeaderComponent {

  @Input() topicName!: string;
  @Input() topicDescription!: string;
  @Input() topicId!: string;
  @Input() classroomName!: string | null;
  classroomUrlFragment!: string;

}
```

2) TopicStorySectionComponent

API:

```
import {Component, Input, OnInit} from '@angular/core';

import {ClassroomDomainConstants} from 'domain/classroom/classroom-domain.constants';
import {TopicViewerDomainConstants} from 'domain/topic_viewer/topic-viewer-domain.constants';
import {UrlInterpolationService} from 'domain/utilities/url-interpolation.service';
import {StorySummary} from 'domain/story/story-summary.model';
import {StoryNode} from 'domain/story/story-node.model';
import {UrlService} from 'services/contextual/url.service';
import {I18nLanguageCodeService} from 'services/i18n-language-code.service';

import './topic-story-section.component.css';

@Component({
  selector: 'topic-story-section',
  templateUrl: './topic-story-section.component.html',
  styleUrls: ['./topic-story-section.component.css'],
})

interface TopicProgressNode {
  practice: TopicPracticeNode[],
  lesson: TopicLessonNode[],
}

export class TopicStorySectionComponent implements OnInit {
  // - Needed because this component renders the story title, lesson list, and completion state.
  // - The parent gets it from canonicalStorySummaries.
  @Input() storyTitle!:string
  @Input() storyDescription!: string

  // to get the study skills url we need them both
  classroomUrlFragment!: string;
  topicUrlFragment!: string;

  // getPracticeCount(): number {
  //   return this.subtopics.filter(
  //     subtopic => subtopic.getSkillSummaries().length > 0
  //   ).length;
  // }

  practiceCount: number = 0; // from service

  lessonCount: number = 0; // from service

  activeTabChange: string="All";

  // Computed in this component from the current story data. This is the
  // ordered list of lesson and practice nodes to pass to the progress strip.
  nodes: TopicProgressNode[] = [];

  // Computed in this component from the current story state. This marks which
  // node is currently active in the progress strip.
```

```
activeNodeId: string = '';
```

3) TopicContentFilterComponent

API:

```
import {Component, EventEmitter, Input, Output} from '@angular/core';

export type TopicContentTab = 'all' | 'lessons' | 'practice';

@Component({
  selector: 'oppia-topic-content-filter',
  templateUrl: './topic-content-filter.component.html',
})
export class TopicContentFilterComponent {
  activeTab: TopicContentTab = 'all';

  @Input() lessonCount!: number;
  @Input() practiceCount!: number;

  @Output() activeTabChange = new EventEmitter<TopicContentTab>();
}
```

4) TopicProgressComponent

API:

```
import {Component, Input} from '@angular/core';

export type ProgressNodeBadgeType = 'none' | 'new' | 'coming-soon';

export interface TopicLessonNode {
  storyNode: StoryNode;
  status: LessonProgressNodeStatus;
}

export interface TopicPracticeNode {
  description: string; // we need to figure this out
  subtopic: Subtopic;
  allSkillsCoveredByLessons: boolean;
  status: PracticeProgressNodeStatus;
}

export type LessonProgressNodeStatus =
  | 'available'
  | 'completed'
  | 'in-progress'
  | 'locked';
```

```

export type PracticeProgressNodeStatus =
  | 'available'
  | 'completed'
  | 'in-progress';

interface BaseProgressNode {
  id: string;
  title: string;
}

export interface TopicLessonProgressNode extends BaseProgressNode {
  status: LessonProgressNodeStatus;
  explorationId: string | null;
  badgeType: ProgressNodeBadgeType;
}

export interface TopicPracticeProgressNode extends BaseProgressNode {
  status: PracticeProgressNodeStatus;
  subtopicId: number;
}

export type TopicProgressNode = TopicLessonProgressNode | TopicPracticeProgressNode;

@Component({
  selector: 'oppia-topic-progress',
  templateUrl: './topic-progress.component.html',
})

export class TopicProgressComponent {

  // Passed from the parent story section component. This is the ordered list
  // of lesson/practice nodes to render in the progress strip.
  @Input() nodes!: TopicProgressNode[];

  // Passed from the parent story section component. This marks which node is
  // currently active so it can be highlighted in the UI.
  activeNodeId!: string;
}

```

5) TopicProgressCircleComponent

API:

```

import {Component, EventEmitter, Input, Output} from '@angular/core';
import {TopicProgressNode} from '../topic-progress.component';

@Component({
  selector: 'topic-progress-circle',
  templateUrl: './topic-progress-circle.component.html',
})

```

```

export class TopicProgressCircleComponent {
  // Passed from the parent progress strip. This is the single node to render.
  @Input() node!: TopicProgressNode;

  // Passed from the parent progress strip. Used to apply active-node styling.
  @Input() isActive = false;

  // Emitted when the user activates this node from the progress strip.
  @Output() nodeSelected = new EventEmitter<string>();
}

```

6) TopicLessonCardComponent

API:

```

import {Component, Input, OnInit} from '@angular/core';

import {AppConstants} from 'app.constants';
import {AssetsBackendApiService} from 'services/assets-backend-api.service';
import {I18nLanguageCodeService} from 'services/i18n-language-code.service';
import {StoryNode} from 'domain/story/story-node.model';
import {ReadOnlyExplorationBackendApiService} from
'domain/exploration/read-only-exploration-backend-api.service';
import {EntityVoiceoversService} from 'services/entity-voiceovers.services';

import './topic-lesson-card.component.css';

/**
 * The four possible states a lesson can be in on the topic page.
 * - not_started : published, never visited
 * - in_progress : published, visited but not fully completed
 * - completed : all checkpoints finished
 * - coming_soon : node is not yet published
 */
export type LessonState =
  | 'not_started'
  | 'in_progress'
  | 'completed'
  | 'coming_soon';

/** Visual state of a single checkpoint node on the progress bar. */
export type CheckpointStatus = 'completed' | 'in_progress' | 'incomplete';

@Component({
  selector: 'topic-lesson-card',
  templateUrl: './topic-lesson-card.component.html',
  styleUrls: ['./topic-lesson-card.component.css'],
})
export class TopicLessonCardComponent implements OnInit {

```

```

//What it can derive from storySummary:

- node for each lesson via storySummary.getAllNodes()
- isCompleted via storySummary.isNodeCompleted(node.getTitle())
- chapterUrl via getChapterUrl(node)
- storyId via storySummary.getId()

/** The StoryNode backing this lesson card. */
@Input() node!: StoryNode;

/** Whether the learner has completed this lesson. */
@Input() isCompleted: boolean = false;

// const visited = this.storySummary.getVisitedChapterTitles() || [];
//   const isVisited = visited.includes(node.getTitle());

@Input() isVisited: boolean = false;

//isNew = node.getFirstPublicationDateMsecs()
//   ? Date.now() - node.getFirstPublicationDateMsecs() < 28 * 24 * 60 * 60 * 1000 &&
//     !isVisited
//   : false;

@Input() isNew: boolean = false;

/** Story ID used to construct the thumbnail asset URL. */
@Input() storyId!: string;

/**
 * Total number of checkpoint nodes to render on the progress bar.
 * Defaults to 3 as a visual affordance on the topic page (per-lesson
 * checkpoint counts are only available inside the player).
 */
@Input() totalCheckpoints: number = 3;

/**
 * How many checkpoints the learner has already completed.
 * On the topic page this is an approximation derived from the lesson
 * state (0 / 1 / totalCheckpoints).
 */
@Input() completedCheckpoints: number = 0;

// Computed in ngOnInit() from the story ID and the node thumbnail filename.
// Remains null if the node does not have a thumbnail.

checkpointStatuses: CheckpointStatus[] = [];

availableTextLanguageCodes: string[] = [];
availableAudioLanguageCodes: string[] = [];

// Currently-selected language codes (set when LanguageSelector emits).

```

```

selectedTextLanguageCode: string | null = null;
selectedAudioLanguageCode: string | null = null;

constructor(
  private readOnlyExplorationBackendApiService: ReadOnlyExplorationBackendApiService,
  private entityVoiceoversService: EntityVoiceoversService,
  private i18nLanguageCodeService: I18nLanguageCodeService,
  /* ... */) {}

```

7) TopicPracticeCardComponent

API:

```

import {Component, Input, OnInit} from '@angular/core';
import {AppConstants} from 'app.constants';
import {Subtopic} from 'domain/topic/subtopic.model';
import {DegreesOfMastery} from 'domain/topic_viewer/read-only-topic.model';
import {TopicViewerDomainConstants} from 'domain/topic_viewer/topic-viewer-domain.constants';
import {UrlInterpolationService} from 'domain/utilities/url-interpolation.service';
import {PracticeSessionPageConstants} from
'pages/practice-session-page/practice-session-page.constants';
import {AssetsBackendApiService} from 'services/assets-backend-api.service';

import './topic-practice-card.component.css';

@Component({
  selector: 'topic-practice-card',
  templateUrl: './topic-practice-card.component.html',
  styleUrls: ['./topic-practice-card.component.css'],
})
export class TopicPracticeCardComponent implements OnInit {

  @Input() subtopic!: Subtopic;
  @Input() degreesOfMastery: DegreesOfMastery = {};
  topicUrlFragment!: string;
  classroomUrlFragment!: string;

  thumbnailUrl: string | null = null;
  thumbnailBgColor: string | null = null;

  /**
   * The derived mastery percentage (0–100) for this subtopic, or
   * `null` if the learner has not yet attempted any skill here.
   */
  masteryPercent: number | null = null;

```

8) TopicPracticeProgressBarComponent

API:

```

import {Component, Input} from '@angular/core';

import './topic-practice-progress.component.css';

@Component({
  selector: 'topic-practice-progress',
  templateUrl: './topic-practice-progress.component.html',
  styleUrls: ['./topic-practice-progress.component.css'],
})
export class TopicPracticeProgressComponent {
  /**
   * The mastery percentage (0–100) to display, or `null` when the learner
   * has not yet attempted any skill in the subtopic.
   */
  @Input() masteryPercent: number | null = null;

```

9) LanguageSelectorComponent

API:

```

import {Component, EventEmitter, Input, OnInit, Output} from '@angular/core';
import {LanguageUtilService} from 'domain/utilities/language-util.service';
import {I18nLanguageCodeService} from 'services/i18n-language-code.service';
import './language-selector.component.css';

export interface LanguageOption {
  code: string;
  label: string;      // Short display name, e.g. "English"
  ariaLabelInEnglish: string;
}

@Component({
  selector: 'language-selector',
  templateUrl: './language-selector.component.html',
  styleUrls: ['./language-selector.component.css'],
})
export class LanguageSelectorComponent implements OnInit {

  //- preferredLanguageCode comes from I18nLanguageCodeService
  - availableTextLanguageCodes comes from the exploration's displayable_language_codes
  - availableAudioLanguageCodes comes from the exploration voiceovers for that chapter

  // - preferredLanguageCode is just the learner's current site language.
  - availableTextLanguageCodes is the list of content languages the exploration can actually display.
  - availableAudioLanguageCodes is the list of voiceover accents available for that exploration.

  @Input() availableTextLanguageCodes!: string[];

```

```

@Input() availableAudioLanguageCodes!: string[];

// Learner's active UI language — shown in "Not available in <X>" helper text.
preferredLanguageCode!: string;

// - This is emitted by LanguageSelectorComponent when the learner chooses a text language.
// - The parent listens to the event and stores the selected value.
@Output() textLanguageSelected = new EventEmitter<string>();

// - Same idea, but for the audio/voiceover language.
// - The child emits null if no audio language is selected.
@Output() audioLanguageSelected = new EventEmitter<string | null>();

// Local arrays derived from the raw codes passed in by the parent.
// These should be built in ngOnInit() using LanguageUtilService.
textLanguageOptions: LanguageOption[] = [];
audioLanguageOptions: LanguageOption[] = [];

// True when there are any audio/voiceover options available.
hasAudioOptions: boolean = false;

constructor(
  private languageUtilService: LanguageUtilService,
  private i18nLanguageCodeService: I18nLanguageCodeService
) {}

```

10) LanguageSelectorModalComponent

API:

```

import {Component, Input, ViewChild} from '@angular/core';
import {NgbActiveModal} from '@ng-bootstrap/ng-bootstrap';
import {ConfirmOrCancelModal} from
'components/common-layout-directives/common-elements/confirm-or-cancel-modal.component';
import {LanguageSelectorComponent} from
'../topic-story-section/topic-lesson-card/language-selector.component';

// Returned to the caller via NgbActiveModal.close(result).
// The caller appends these to the chapter URL before navigating.
export interface LessonLanguageSelection {
  textLanguageCode: string;
  audioLanguageCode: string | null;
}

@Component({
  selector: 'language-selector-modal',

```

```

    templateUrl: './language-selector-modal.component.html',
  })
  export class LanguageSelectorModalComponent extends ConfirmOrCancelModal

  // Set by the opener: modalRef.componentInstance.preferredLanguageCode = ...
  @Input() preferredLanguageCode!: string;
  @Input() availableTextLanguageCodes!: string[];
  @Input() availableAudioLanguageCodes!: string[];

```

Responsiveness, accessibility, RTL, and i18n are treated as core acceptance criteria, not polish. The redesign will explicitly reuse topic-page patterns from [core/templates/pages/topic-viewer-page/topic-viewer-page.component.css](#), [core/templates/pages/topic-viewer-page/stories-list/topic-viewer-stories-list.component.css](#), [core/templates/pages/topic-viewer-page/topic-viewer-page.component.html](#), and [core/templates/components/summary-tile/story-summary-tile.component.html](#), then apply the same standards to all new card/strip/modal components.

For responsiveness (all device sizes), the new UI will keep Oppia's breakpoint strategy (mobile/tablet/desktop), use fluid grid/flex layouts, avoid fixed-width interaction blocks, and verify behavior across narrow and wide layouts for all key states (All/Lessons/Practice filters, locked cards, progress strip, and language modal). Acceptance requires no overflow clipping of interactive controls, no unreachable CTAs, and consistent focus order regardless of viewport size.

For accessibility (Lighthouse 100), we define a measurable scope: [Lighthouse accessibility](#) audit on the redesigned topic page for a logged-in learner baseline on both desktop and mobile profiles. All interactive elements will be semantic controls (button/link, not clickable div), keyboard reachable, and visibly focusable; stateful UI (progress, locked/disabled, selected) will include ARIA labels and state announcements; color contrast and non-color status cues will be enforced. This gives a reviewable and objective "100" target instead of an informal claim.

(Reference- <https://github.com/oppia/oppia/wiki/Testing-for-Accessibility>)

For RTL support, new components will follow the same direction-aware patterns already used in topic/story templates via [i18nLanguageCodeService](#): mirrored directional icons, RTL-safe spacing/layout logic, and removal of hardcoded left/right assumptions in component styling. RTL validation will be done against the same interaction paths as LTR to ensure functional parity, not just visual mirroring.

For full internationalization, every user-facing string in redesigned components will be translation-key based (with placeholders where needed), with no runtime sentence concatenation that can break grammar in localized languages. Existing fallback behavior for translated topic/classroom metadata will be preserved, and new keys will be added through the standard [Oppia's i18n pipeline](#) so all states (including locked, progress, language prompts, and CTA variants) are localizable.

Evidence PRs (understanding of these areas)

- <https://github.com/oppia/oppia/pull/21495>
- <https://github.com/oppia/oppia/pull/21442>
- <https://github.com/oppia/oppia/pull/23328>
- <https://github.com/oppia/oppia/pull/21330>
- <https://github.com/oppia/oppia/pull/21334>

Documentation changes

N/A

Metrics Plan

Reference used - [PRD](#)

Event (see PRD)	Event parameters (see PRD)	Do we already record the event + parameters? • If so, please link to the corresponding code on GitHub. • If not, describe the changes needed to do so.
Learner clicks Start Lesson on a lesson card from the Topic Page	topic_id, story_id, exploration_id, selected_content_language_code, preferred_site_language_code	Lesson start events are already recorded in the exploration player analytics pipeline. However, the launch context from the Topic Page (selected content language vs preferred site language) is not currently captured. To support the PRD metric, we will add a topic_page_lesson_start event in the lesson-card Start handler before navigation to the lesson player. This event will include the selected language and preferred language so we can measure learners starting lessons in unfamiliar languages.
Learner exits a lesson before reaching the second card	exploration_id, selected_content_language_code, preferred_site_language_code, exit_stage	Lesson exit events are partially recorded in the exploration player, but early exit before reaching the second card is not explicitly tracked in relation to the Topic Page launch context. We will extend the exploration-player analytics to record lesson_early_exit events when the learner exits before the second card. The event will include the language parameters passed from the Topic Page.
Learner begins a practice session from the Topic Page	topic_id, exploration_id, skill_ids, completion_state	Lesson completion events are already recorded in the exploration player. No new event needs to be introduced. The Topic Page redesign will reuse these existing completion events and associate them with topic metadata to compute the PRD metric.
Learner completes a lesson introducing a skill	topic_id, exploration_id, skill_ids, completion_state	Lesson completion events are already recorded in the exploration player. No new event needs to be introduced. The Topic Page redesign will reuse these existing completion events and associate them with topic metadata to compute the PRD metric.
Learner returns and starts another practice session	topic_id, subtopic_id, practice_attempt_number	Practice attempts are currently recorded, but the attempt number per learner within a topic is not explicitly tracked in analytics. We will extend the practice start event to include a practice_attempt_number parameter that increments for each attempt. This allows

		measurement of repeat engagement (second, third, fourth practice sessions).
--	--	---

Implementation Plan

Milestone Table (include both PRs and other actions that need to be taken prior to launch)

Milestone 1

Key objective for this milestone: Implement a basic version of the updated topic player page (based on these mocks), and ensure that it is fully responsive for desktop and mobile portrait views. This involves the following steps:

- 1) Display the topic name and description prominently, along with the associated story name and description.
- 2) Include a link to the study skills page from the topic page, ensuring the page is accessible and properly styled.
- 3) Filterable, scrollable sequence of lessons and practices.
 - Displayed the count of lessons and practices
 - Practices should be interspersed in the correct order
 - Correctly suggest what item to work on next. This suggestion should be the default when the page loads and indicated with a green arrow.
 - Dynamic scroll arrow visibility
 - Correct scroll distance
 - Selecting a lesson or practice shows details and links to start, resume, study, or practice, as appropriate
 - Filter status is remembered the next time the user loads the page
- 4) Maintain a uniform UI consistent with the topic editor page and ensure full responsiveness for both desktop and mobile portrait views.

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1.1	Integrate redesigned Topic Viewer feature flag, container setup, and initialize redesigned component architecture Goal: Verify that the <code>RedesignedTopicViewerPage</code> feature flag	None	May 25	May 27

	introduced in PR #20406 is correctly wired into the Topic Viewer flow, while integrating feature-flag-based conditional rendering between the redesigned Topic Viewer UI and the existing legacy Topic Page UI. Preserve existing route handling, auth validation, and topic data-fetching behavior. Additionally, reorganize the <code>topic-viewer-page/</code> directory. Completion: In <code>topic-viewer-page.component</code>, add feature-flag-based rendering logic and initialize the redesigned Topic Viewer container structure while preserving existing topic-page behavior. Verify that the <code>RedesignedTopicViewerPage</code> feature flag is correctly integrated into the Topic Viewer flow and add integration placeholders for redesigned components without affecting legacy rendering flows. Refactor the Topic Viewer folder structure and create the folder named as <code>deprecations/</code> for deprecated files			
1.2	Implement redesigned topic header layout Goal: Implement redesigned topic-header UI displaying breadcrumb navigation, topic title, and topic description according to mocks. Ensure responsive layout support and semantic accessibility structure. Completion: Create <code>TopicHeaderComponent</code> and integrate responsive topic-header rendering into the redesigned Topic Viewer layout with semantic heading hierarchy and accessible breadcrumb navigation.	1.1	May 27	May 29
1.3	Implement redesigned story-section layout and Study Skills CTA Goal: Implement story-section rendering with story title, story description, lesson/practice counts, and Study Skills CTA integration. Ensure multiple stories render correctly in responsive layouts. Completion: Create <code>TopicStorySectionComponent</code>, integrate story rendering loop into the redesigned Topic Viewer layout, and add responsive Study Skills	1.2	May 29	June 2

	CTA behavior with accessibility semantics.			
1.4	Implement redesigned lesson-card rendering system Goal: Implement redesigned lesson-card UI with lesson metadata, lesson-state visuals, and responsive layouts for desktop/mobile portrait views. Completion: Create <code>TopicLessonCardComponent</code> , integrate lesson-card rendering into story sections, and ensure lesson CTA rendering behaves consistently across responsive layouts.	1.3	June 2	June 5
1.5	Implement redesigned practice-card rendering system Goal: Implement redesigned practice-card UI with practice metadata, CTA states, and responsive rendering behavior according to mocks. Completion: Create <code>TopicPracticeCardComponent</code> and integrate practice-card rendering into story sections.	1.3,1.4	June 5	June 8
1.6	Create lesson-progress rendering infrastructure Goal: Prepare lesson-progress rendering architecture and integrate learner-progress metadata into redesigned Topic Viewer lesson-card flows. Display lesson progress bars and checkpoint progression indicators on redesigned lesson cards according to mocks. Completion: Add lesson-progress state mapping utilities and prepare normalized progress-node data for redesigned lesson-card rendering without yet implementing final progress visuals. Integrate progress rendering reuse <code>oppia-checkpoint-bar</code> into <code>TopicLessonCardComponent</code> and implement responsive lesson-progress visuals using learner progress metadata and accessibility-compliant progress semantics.	1.4	June 14	June 17
1.7	Backend: Arc data model + Creator arc UI + API Goal: Add Arc domain class, schema v7 on-read migration, backend validation, story-editor	-	June 14	June 18

	split-line UI, topic viewer API arcs field, skill-based practice endpoint, and access validator Completion: One PR with all backend + creator-side changes. On-read migration handles existing data without a Beam job.			
1.8	Migration PR(beam job needs to be written for story model)	1.7	June 18	June 22
1.9	Implement reusable language-selection dropdown component Goal: Implement reusable secondary-language-selection dropdown UI for lessons requiring explicit language selection. Implement modal flows for lessons unavailable in the learner's preferred language and enforce language-selection behavior before lesson launch. Completion: Create <code>LanguageSelectorComponent</code> with text/audio language-selection behavior, validation handling, dropdown accessibility semantics, and responsive rendering support.	1.4	June 15	June 19
1.10	Implement completed-lesson Resume/Start over modal behavior Goal: Extend lesson-action modal flows to support completed-lesson review behavior (<code>Resume / Start Over</code>) according to mocks. Completion: Add completed-lesson orchestration logic and synchronize modal CTA behavior with lesson completion states and lesson-launch flows.	1.6	June 17	June 20
1.11	Polish Creator side UI	1.7	June 20	June 23
–	At this point the redesigned Topic Page layout is fully implemented behind the feature flag. The following functionality is working: • Topic header and story information display • Lesson and practice cards rendered in the correct order • Study Skills navigation			

	• Lesson progress indicators and checkpoint progress rendering • Arc-based story structure support and Topic Viewer API integration • Creator-side Arc management UI and story splitting workflows • Story model migration support • Secondary language selection and lesson-launch flows • Completed lesson Resume / Start Over modal behavior • Responsive desktop and mobile layouts with accessibility support			
-	<u>Demo to PMs(80+% of M1 done):</u> 1. The overall Topic Page redesign has been implemented. 2. Clicking the Study Skills link, either from the story header or from a practice card, leads to the Study Skills page. From there, selecting a concept leads to the concept revision page. 3. Lessons and practices are shown in the correct order and can be selected to display additional details. 4. Lesson progress indicators and checkpoint progress rendering are displayed correctly according to learner progress. 5. Arc creation and management workflows are functional from the creator side, and Arc data is exposed correctly through the Topic Viewer API. 6. Existing stories continue to function correctly through migration support. 7. Languages are selected correctly for lessons, including secondary language selection and language-aware lesson launching flows. 8. Users are shown Resume / Start Over modal flows when opening completed lessons. 9. All applicable frontend, backend, and acceptance tests have been written and pass robustly.	1.11	June 20	
-	Milestone 1 bug fixes and review iterations Goal: Resolve CI failures, flaky tests, accessibility edge cases, responsive-layout inconsistencies, and mentor/reviewer feedback. Completion: Perform review-driven iterations	1.11	June 27	July 3

	and stabilize redesigned Topic Viewer interactions, accessibility behavior, and responsive layouts across all Milestone 1 functionality.			
–	Milestone 1 revision buffer period Goal: Final stabilization period before midpoint evaluation including cleanup, accessibility verification, etc	1.11	July 4	July 10

Milestone 2

Key objective for this milestone:

- 1) Show progress indicators on lessons and practice sessions
- 2) Show pop-ups when a user returns to an in-progress or completed lesson
- 3) Implement the “Representation of Chapter Availability” section in the mocks by updating the existing "coming soon" UI and new tag on the chapters.
- 4) Support secondary language selection, enforced with pop-ups when necessary
- 5) Persist secondary language choices. At this point, the full language selection algorithm should be implemented

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
2.1	Implement scrollable lesson/practice progress strip also implement scroll controls and dynamic arrow visibility logic Goal: Implement horizontally scrollable lesson/practice strip with correct lesson/practice ordering, responsive overflow behavior, and keyboard-operable node navigation. Add left/right scroll-arrow controls, dynamic scroll-arrow visibility logic, and correct scroll-distance handling across responsive layouts. Completion: Create TopicProgressComponent and TopicProgressCircleComponent, integrate progress-strip rendering into story sections, and implement responsive scrolling behavior for desktop and mobile portrait layouts. Implement responsive scroll orchestration, dynamic arrow-visibility behavior, and scroll-distance	Milestone 1	July 18	July 22

	calculations inside <code>TopicProgressComponent</code> .			
2.2	Implement suggested next-activity indicator orchestration Goal: Highlight the learner's recommended next lesson/practice using the green-arrow indicator shown in mocks and ensure the recommended node is selected by default on page load. Completion: Add learner-progression orchestration inside <code>TopicStorySectionComponent</code> and synchronize recommended progress-node selection with lesson/practice-card rendering behavior.	2.1	July 22	July 25
2.3	Integrate serial chapter launch state infrastructure Goal: Prepare serial-launch state orchestration for redesigned Topic Viewer cards and progress-strip rendering flows. Completion: Integrate serial-launch metadata into Topic Viewer normalization utilities and prepare state propagation for <code>coming_soon</code> and <code>new</code> lesson states(Referring newly written CUJ'S)	2.1	July 22	July 25
2.4	Implement all backend changes needed to show practice in lesson card via acquired skills, in arc test via acquired skills in all lessons of the Arc, and via acquired skills taught in the whole topic at the end i.e the Mastery Practice test	2.3	July 20	July 25
2.5	Make necessary changes in lesson card component with new design as per the mocks	2.4	July 26	July 29
2.6	Reuse Practice Card component as Arc test component	2.4	July 27	July 30
2.7	Implement Topic Mastery Practice test component i.e the whole topic practice test at the end	2.4	July 27	July 31
2.8	Reformat the whole topic page as per the updated mocks	2.7	Aug 1	Aug 5
2.9	Implement consistency of the Topic-editor with the redesigned topic page	2.8	Aug 6	Aug 10

	Goal: Ensure the Topic Editor UI, especially the preview tab, remains visually and functionally consistent with the redesigned Topic Page experience. Completion: Updated the Topic Editor interfaces, particularly the Preview tab, to align visually and functionally with the redesigned Topic Page experience while preserving existing editor workflows and functionality.			
2.10	Implement acceptance-test coverage for CUJ'S Goal: Implement acceptance-test coverage for redesigned Topic Viewer learner flows related to lesson progress indicators, practice mastery indicators, progress-strip behavior, lesson-selection flows, and modal orchestration. Completion: Add acceptance tests covering CUJ's. Ensure all tests pass reliably in CI.	2.9	Aug 10	Aug 18
–	Annotation: At this stage the redesigned Topic Page fully supports learner progress tracking and language-aware lesson launching. The following functionality is implemented: • Lesson progress indicators • Practice mastery indicators • Resume / Review / Start Over flows • Serial chapter launch UI states • Secondary language selection and persistence • Full CUJ acceptance tests			
–	<u>Demo to PMs(80+% of M2 done):</u> 1. Progress indicators are displayed, and progress is calculated correctly, for both lessons and practices. 2. Users are shown pop-ups when loading in-progress or completed lessons. 3. Languages are selected correctly for lessons. This includes secondary language selection and pop-ups. 4. All applicable frontend, backend, and acceptance tests have been written and pass robustly.	2.1-2.16	Aug 16	
–	Milestone 2 bug fixes and review iterations	2.16	Aug 20	Aug 26

	Goal: Resolve testing issues, CI failures, modal edge cases, language-resolution inconsistencies, accessibility issues, and reviewer feedback. Completion: Perform review-driven stabilization across learner-progress flows, modal orchestration, language persistence behavior, accessibility support, and acceptance-test reliability.			
–	Final revision buffer period before final evaluation Goal: Final stabilization and cleanup period before final evaluation including accessibility verification, documentation refinement, responsive-layout verification, etc.	2.17	Aug 27	Sept 2

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Need sync between Android / Web?	Key stakeholders
1.	RedesignedTopicViewerPage	Enables the redesigned Topic Page experience on the web (new layout, unified lesson/practice flow, redesigned cards/strip behavior). When off, the legacy Topic Page experience is shown.	No	LEAP Team
2.	SerialChapterLaunchLearnerView	Enables serial chapter launch behavior for learners on Topic Page (only released lessons are actionable, locked lessons show Coming Soon state, release progression UX is enforced).	No	LEAP Team

Future Work

Practice Session Experience

The redesigned Topic Page integrates lessons, practice sessions, and study resources into a unified learner journey. However, the Practice Session Experience described in the PRD will be implemented in future work.

Potential future improvements include:

1. Enhancements to the practice session player UI and navigation
2. Improved mastery visualization and feedback during practice
3. Adaptive practice recommendations based on learner progress
4. Expanded language support for practice sessions

The Topic Page redesign implemented in this project prepares the foundation for these improvements by integrating practice cards, exposing mastery progress, and linking lessons directly to relevant practice sessions.